

香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

Operating Systems
Lecture 10

Modern computer systems: concurrency

Prof. Yunming XIAO

School of Data Science

Table of contents

- Modern computer system research venues
- Concurrency issues today
  - [Extended reading] Understanding Real-World Concurrency Bugs in Go, **ASPLOS'19**
- Case study: systems for machine learning/AI
  - **Basics**: neural network, data parallelism, model parallelism
  - **PipeDream**: Generalized Pipeline Parallelism for DNN Training, **SOSP'19**
  - **Orca**: A Distributed Serving System for Transformer-Based Generative Models, **NSDI'22**
  - **DistServe**: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving, **OSDI'24**

Table of contents

- Modern computer system research venues
- Concurrency issues today
  - [Extended reading] Understanding Real-World Concurrency Bugs in Go, **ASPLOS'19**
- Case study: systems for machine learning/AI
  - **Basics**: neural network, data parallelism, model parallelism
  - **PipeDream**: Generalized Pipeline Parallelism for DNN Training, **SOSP'19**
  - **Orca**: A Distributed Serving System for Transformer-Based Generative Models, **NSDI'22**
  - **DistServe**: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving, **OSDI'24**

Computer system research venues

- In CS, conference publication is a major, often primary, way to disseminate research

- Core OS conferences
  - **SOSP**: ACM Symposium on Operating Systems Principles (started from 1967)
  - **OSDI**: USENIX Symposium on Operating Systems Design and Implementation (started from 1994)
  - **EuroSys**: ACM European Conference on Computer Systems (started from 2006)
  - **ATC**: USENIX Annual Technical Conference (started from 1975, discontinued 2025, reassumed to be ACM SIGOPS ATC from 2026)

Computer system research venues

- Related fields
  - [Distributed System] NSDI / SoCC / Middleware
  - [Network] SIGCOMM / CoNEXT
  - [Architecture] ASPLOS / ISCA / MICRO
  - [Programming Language] POPL / PLDI / OOPSLA
  - [Security] IEEE S&P / CCS / USENIX Security / NDSS
  - [Storage] FAST

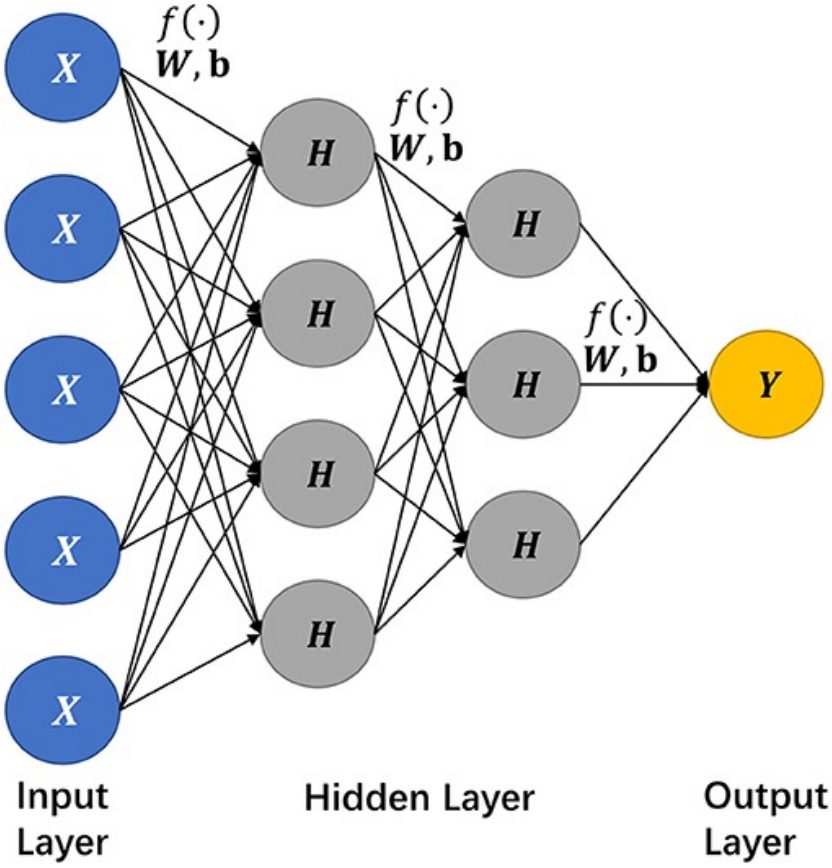
Table of contents

- Modern computer system research venues
- Concurrency issues today
  - [Extended reading] Understanding Real-World Concurrency Bugs in Go, **ASPLOS'19**
- Case study: systems for machine learning/AI
  - **Basics**: neural network, data parallelism, model parallelism
  - **PipeDream**: Generalized Pipeline Parallelism for DNN Training, **SOSP'19**
  - **Orca**: A Distributed Serving System for Transformer-Based Generative Models, **NSDI'22**
  - **DistServe**: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving, **OSDI'24**

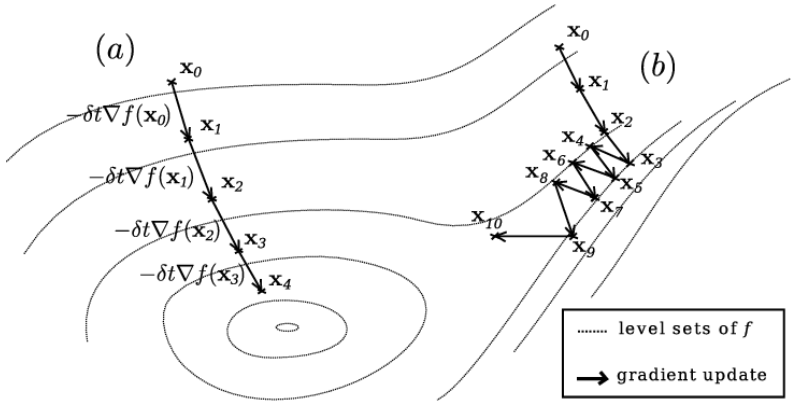
Table of contents

- Modern computer system research venues
- Concurrency issues today
  - [Extended reading] Understanding Real-World Concurrency Bugs in Go, **ASPLOS'19**
- Case study: systems for machine learning/AI
  - **Basics**: neural network, data parallelism, model parallelism
  - **PipeDream**: Generalized Pipeline Parallelism for DNN Training, **SOSP'19**
  - **Orca**: A Distributed Serving System for Transformer-Based Generative Models, **NSDI'22**
  - **DistServe**: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving, **OSDI'24**

Neural network



Training a neural network: gradient descent

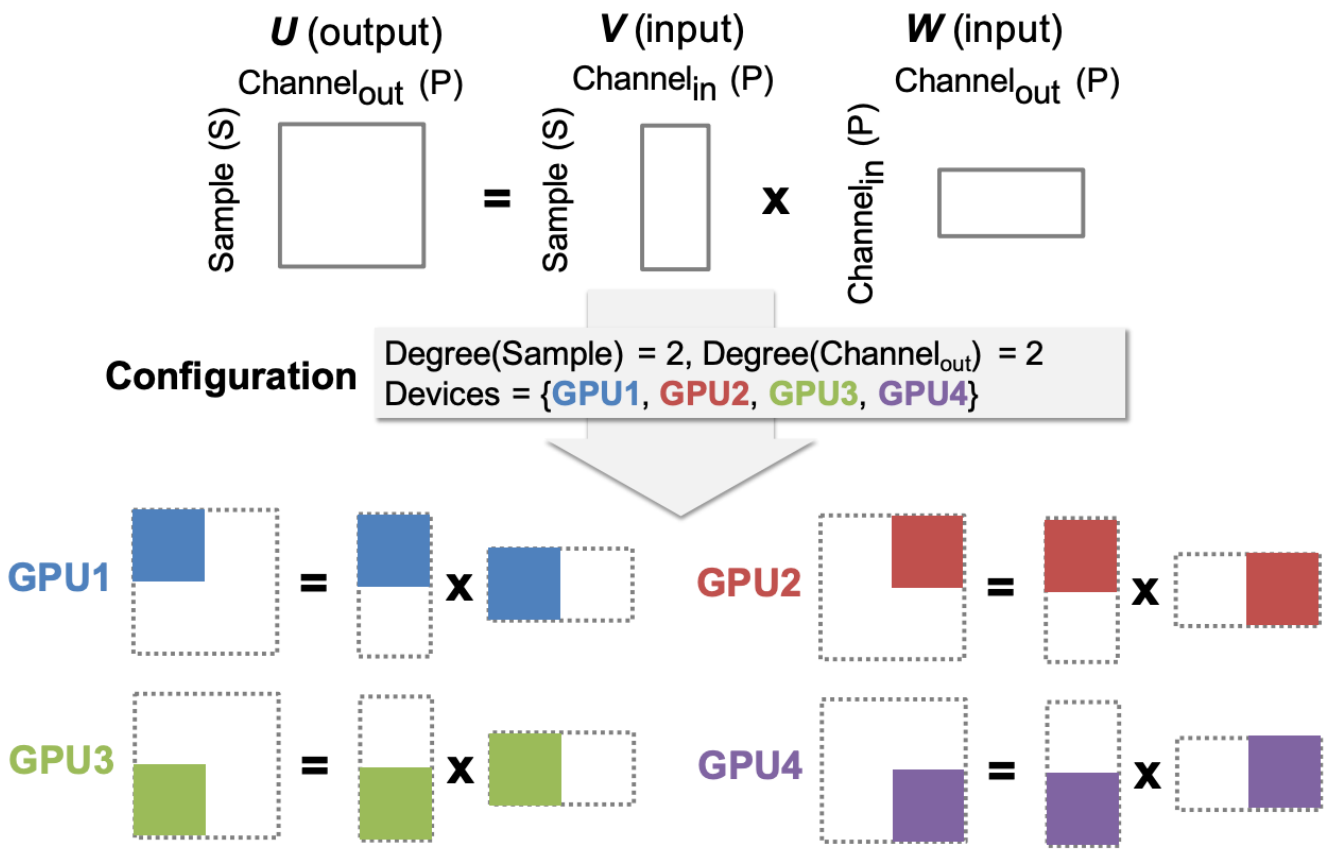


- General Form
- Objective: to find the minimum of differentiable function $f(W)$, starting from W_0

- Method: to iterate calculation as

$$W_{i+1} = W_i - \alpha \, \nabla f(W_i)$$

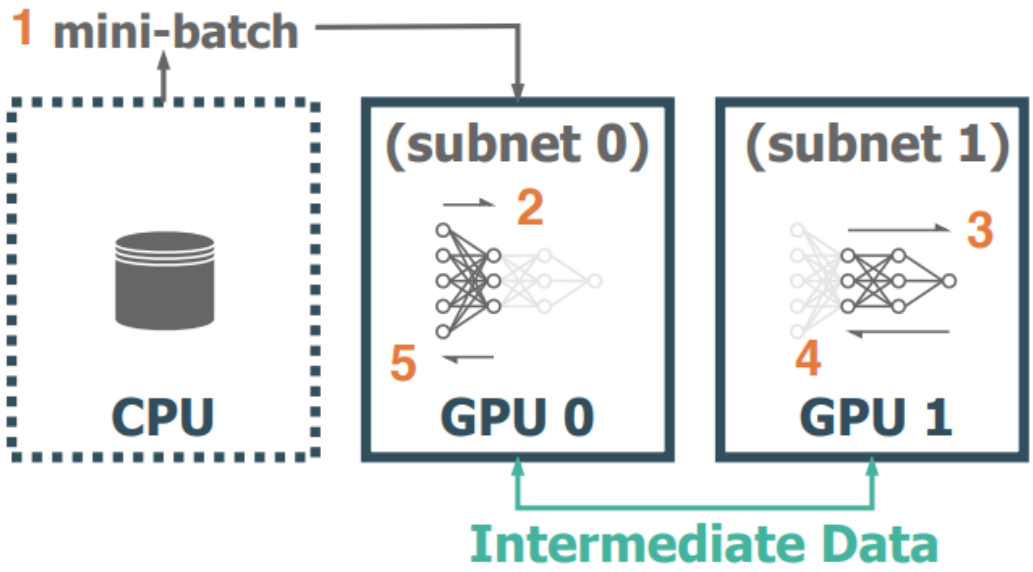
Data parallelism



Data parallelism



Model parallelism (for Deep Neural Network)



Chen et al. "Efficient and Robust Parallel DNN Training through Model Parallelism on Multi-GPU Platform". In ArXiv, 2016.

Table of contents

- Modern computer system research venues
- Concurrency issues today
  - [Extended reading] Understanding Real-World Concurrency Bugs in Go, **ASPLOS'19**
- Case study: systems for machine learning/AI
  - **Basics**: neural network, data parallelism, model parallelism
  - **PipeDream**: Generalized Pipeline Parallelism for DNN Training, **SOSP'19**
  - **Orca**: A Distributed Serving System for Transformer-Based Generative Models, **NSDI'22**
  - **DistServe**: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving, **OSDI'24**

PipeDream: Generalized Pipeline Parallelism for DNN Training

Deepak Narayanan[§], Aaron Harlap[†], Amar Phanishayee[★], Vivek Seshadri[★], Nikhil R. Devanur[★], Gregory R. Ganger[†], Phillip B. Gibbons[†], Matei Zaharia[§]

[★] **Microsoft Research** [†] **Carnegie Mellon University** [§] **Stanford University**



Deep Neural Networks have empowered state of the art results across a range of applications…



Image Classification

வணக்கம் என் பெயர் தீபக்

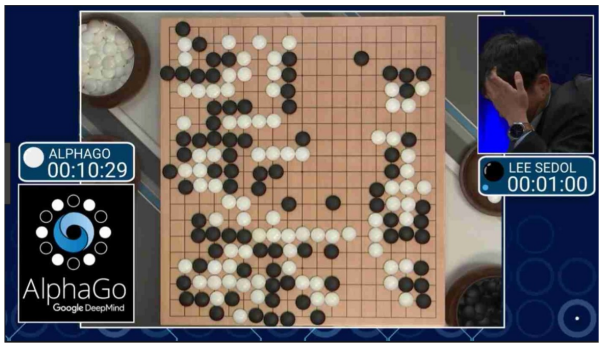


Hello, my name is Deepak

Machine Translation

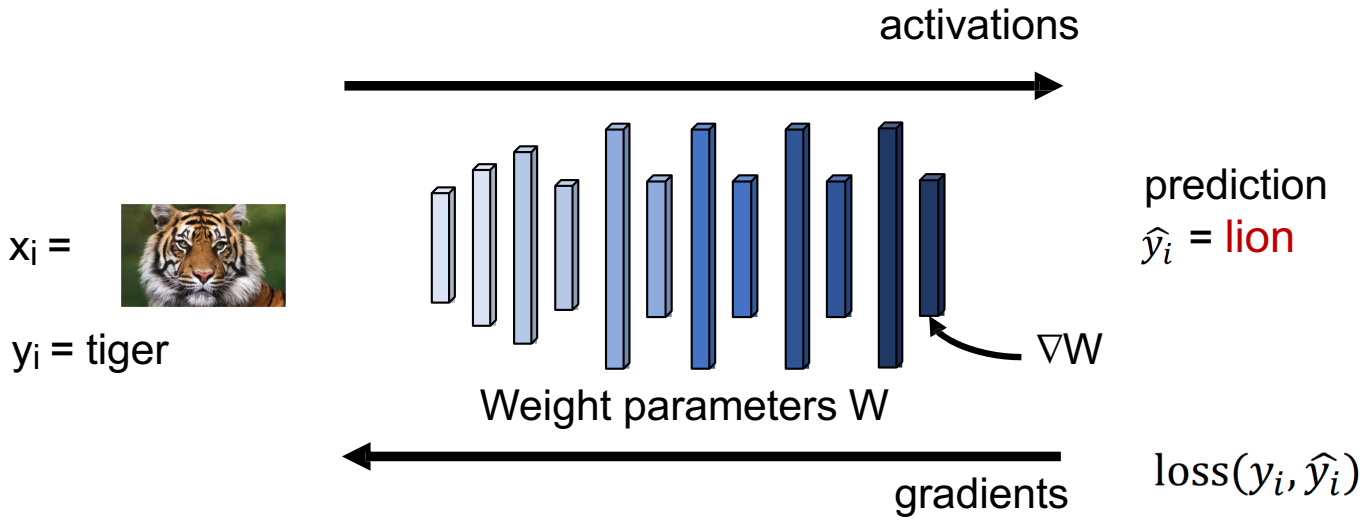


Speech-to-Text



Game Playing

…but first need to be trained!



W optimized using standard iterative optimization procedures

$$W = W - \eta \cdot \nabla W$$

Background: DNN Training



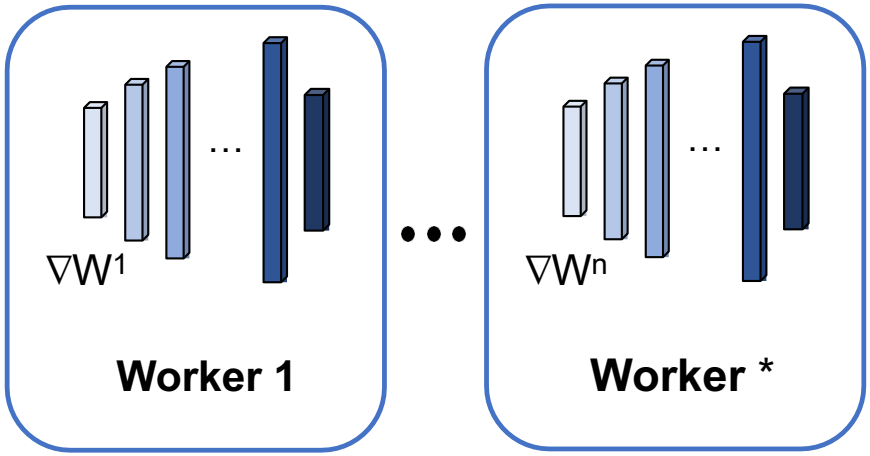
Model training time- and compute- intensive!

Parallelizing DNN Training: Data Parallelism



) copies of the same model

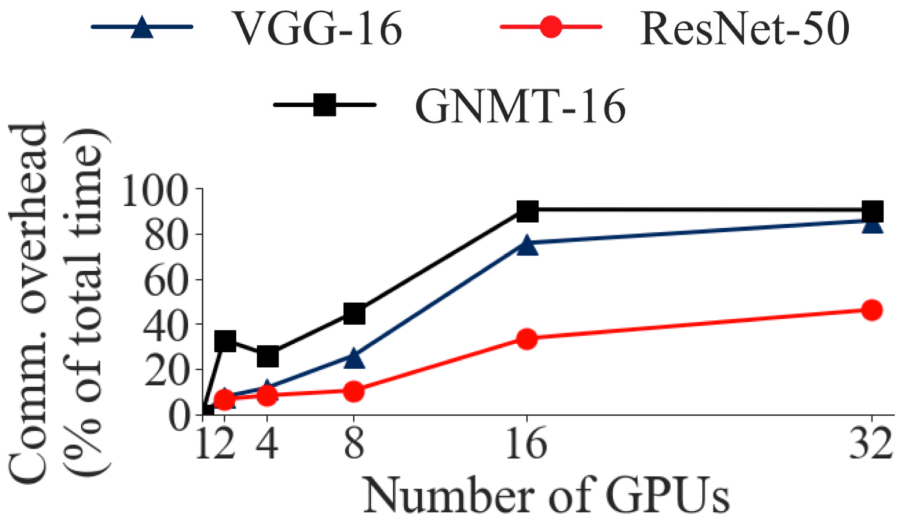




$\nabla W = \nabla W^1 + \nabla W^2 + \cdots + \nabla W^n$

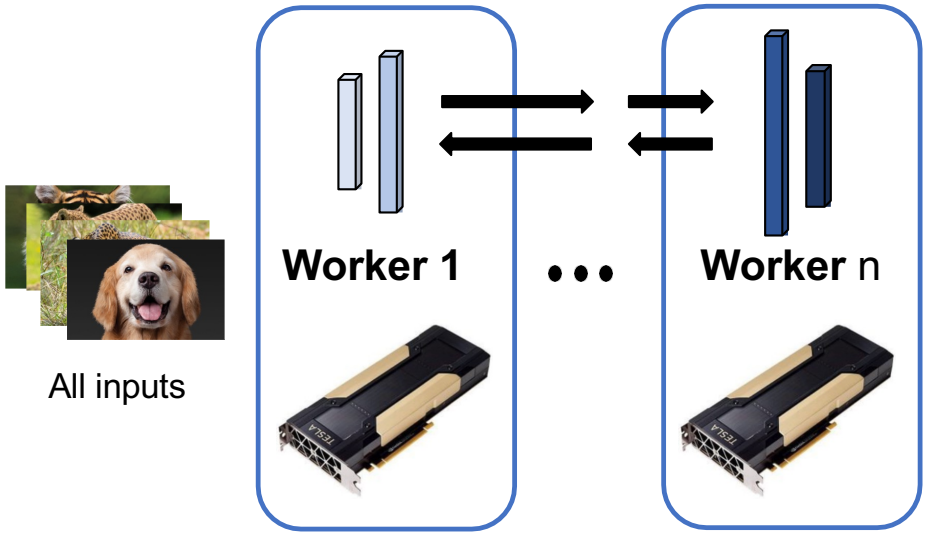
Gradient aggregation using AllReduce

Despite many performance optimizations, communication overhead high!



8xV100s with NVLink (AWS)
PyTorch + NCCL 2.4

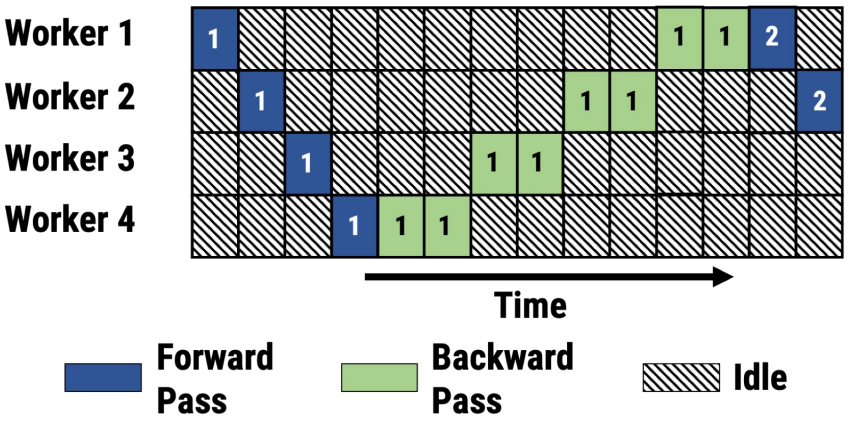
Parallelizing DNN training: Model Parallelism



Single version of weights split over workers

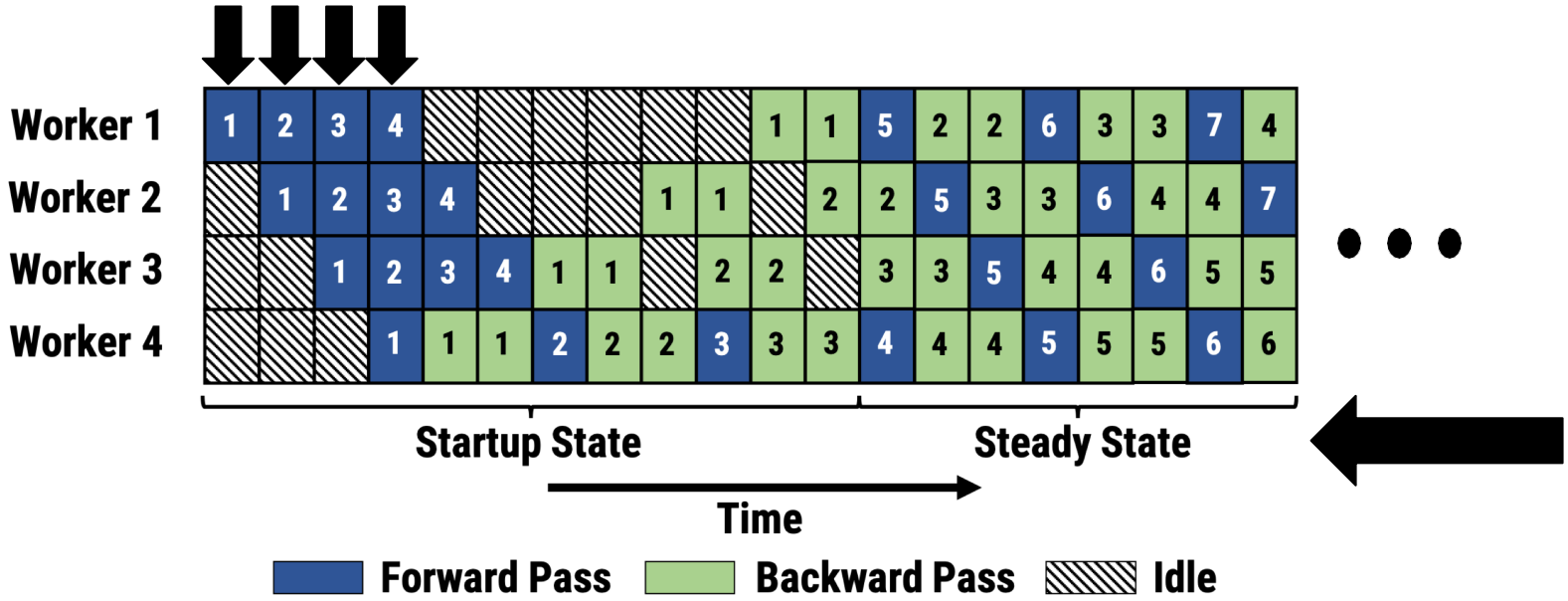
Activations and gradients sent between workers using peer-to-peer communication

Low hardware efficiency



PipeDream: Pipeline-Parallel Training

We propose **pipeline parallelism**, a combination of data and model parallelism with pipelining



Pipeline-parallel training up to **5.3x faster** than data parallelism
without sacrificing on final accuracy of the model

Another look at the pipeline parallelism



Pipelining in DNN Training != Traditional Pipelining

- How should the operators in a DNN model be partitioned into pipeline stages?
  - Each operator has a **different computation time**
  - Activations and gradients need to be **communicated** across stages

- How should forward and backward passes of different inputs be scheduled?
  - Training is **bidirectional**
  - Forward pass followed by backward pass to compute gradients

- How should weight and activation versions be managed?
  - Backward pass operators depend on **internal state** (W, activations)

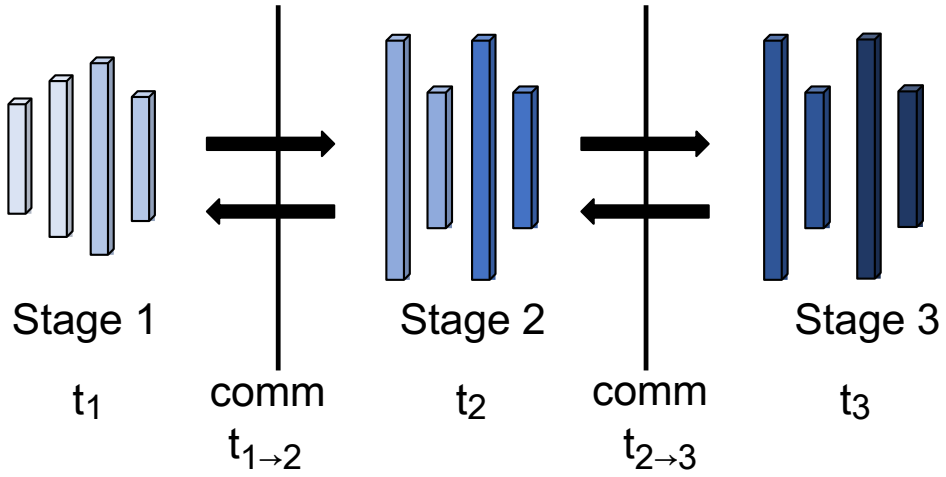
Outline

- Background and Motivation

- **Challenges for effective pipeline-parallel training**
  - **Partitioning and load balancing operators across workers**
  - Scheduling of forward and backward passes of different inputs
  - Managing weights and activation versions for effective learning

- Evaluation

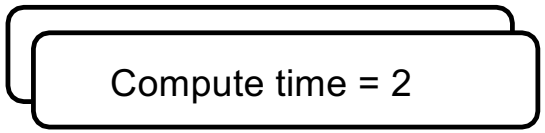
How do we assign operators to pipeline stages?



- Desiderata #1: t_1, t_2, t_3 as close to each other as possible
  - Compute resources seldom idle → better hardware efficiency

- Desiderata #2: $t^{comm}_{1\to2}$ and $!^{comm}_{2\to3}$ minimized
  - Less communication → better hardware efficiency

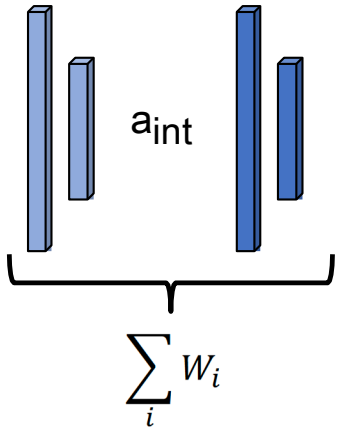
How do we assign operators to pipeline stages?



Throughput =
(1 / 2) × 2 = 1



Throughput = 1



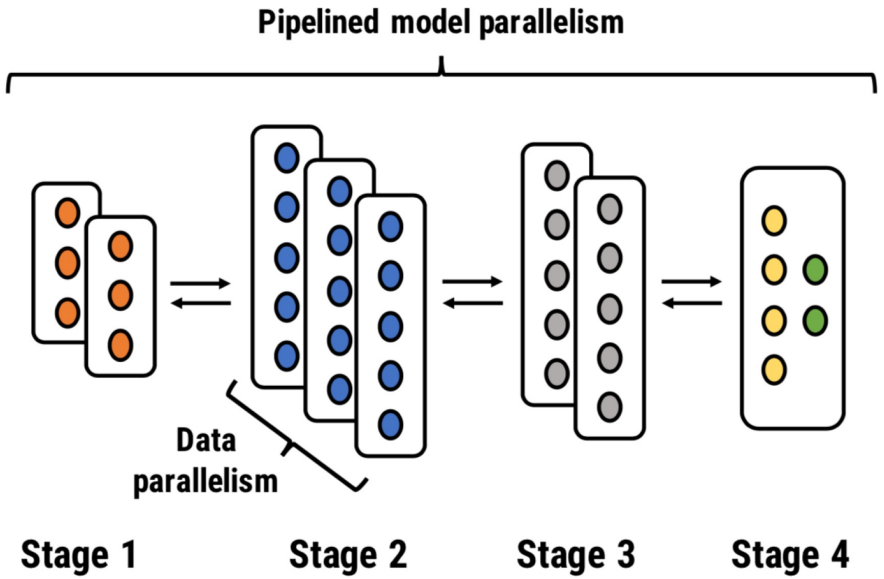
For **some** operators,
$\sum_i W_i < 2a_{int}$

Better load balancing across stages

Data-parallel communication is small

Replication of stages helps load balance computation and reduce communication between workers

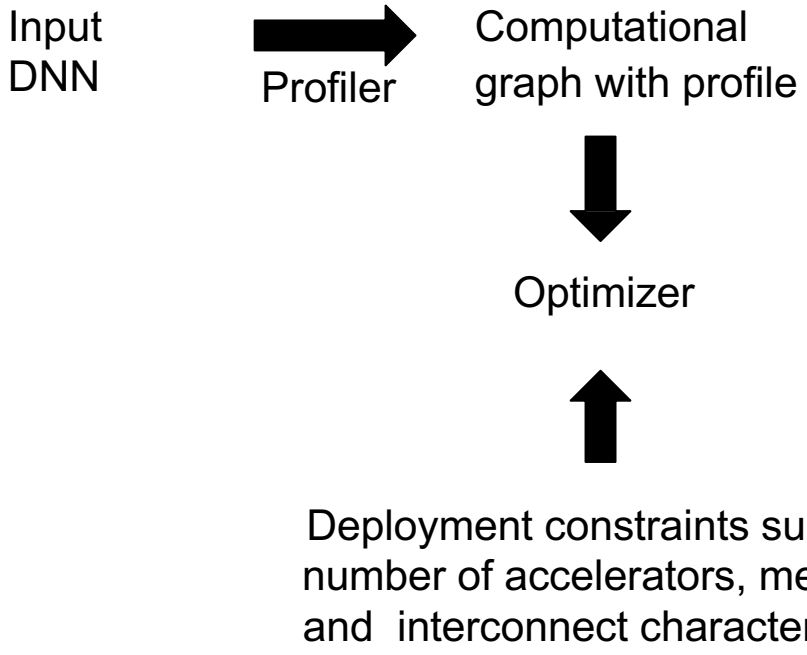
Example PipeDream configuration



Configuration: 2-3-2-1

Stages can have different replication factors

PipeDream Profiler and Optimizer



Determines a partitioning of operators amongst workers, while also deciding replication factors

Generalizes along many axes

- Hardware topologies
- Model structures
- Memory capacities of workers

See paper for details of algorithm!

Outline

- Background and Motivation

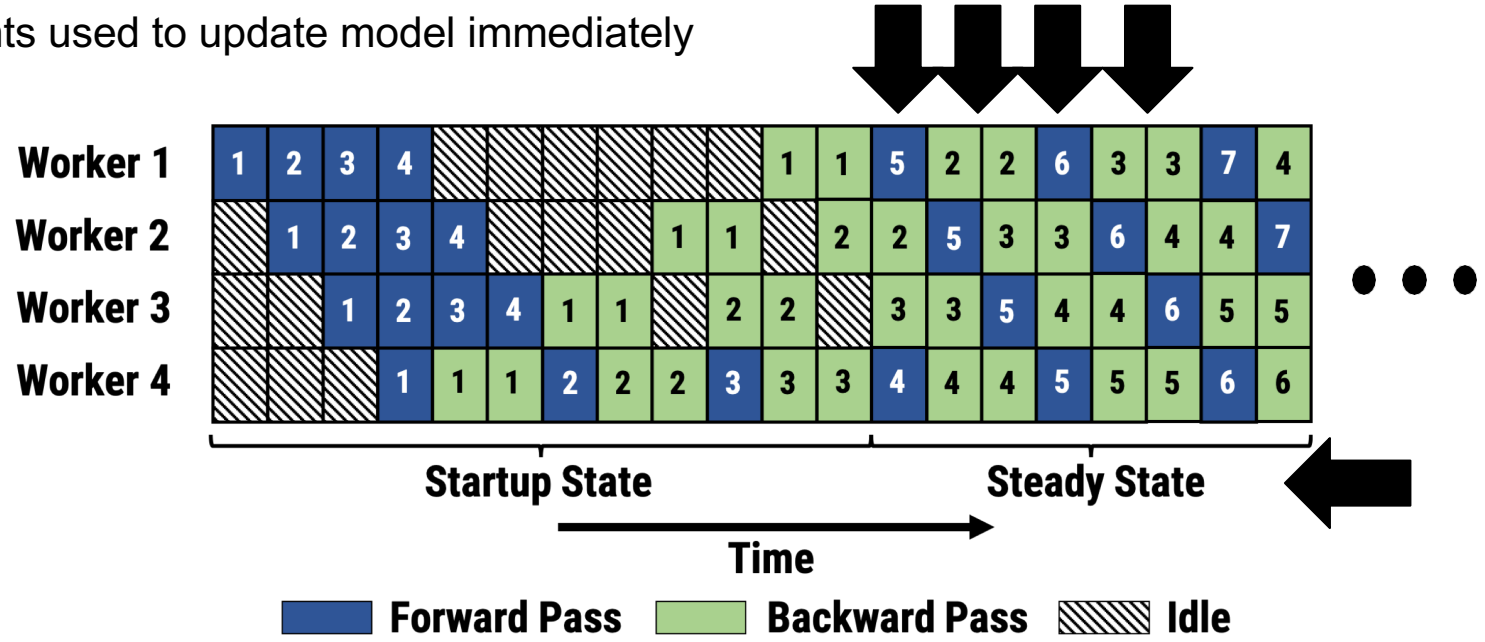
- **Challenges for effective pipeline-parallel training**
  - Partitioning and load balancing operators across workers
  - **Scheduling of forward and backward passes of different inputs**
  - Managing weights and activation versions for effective learning

- Evaluation

1F1B Scheduling

Workers **alternate** between forward and backward passes

- Workers always utilized
- Gradients used to update model immediately



To support stage replication, need to modify this mechanism slightly – see paper for details!

Outline

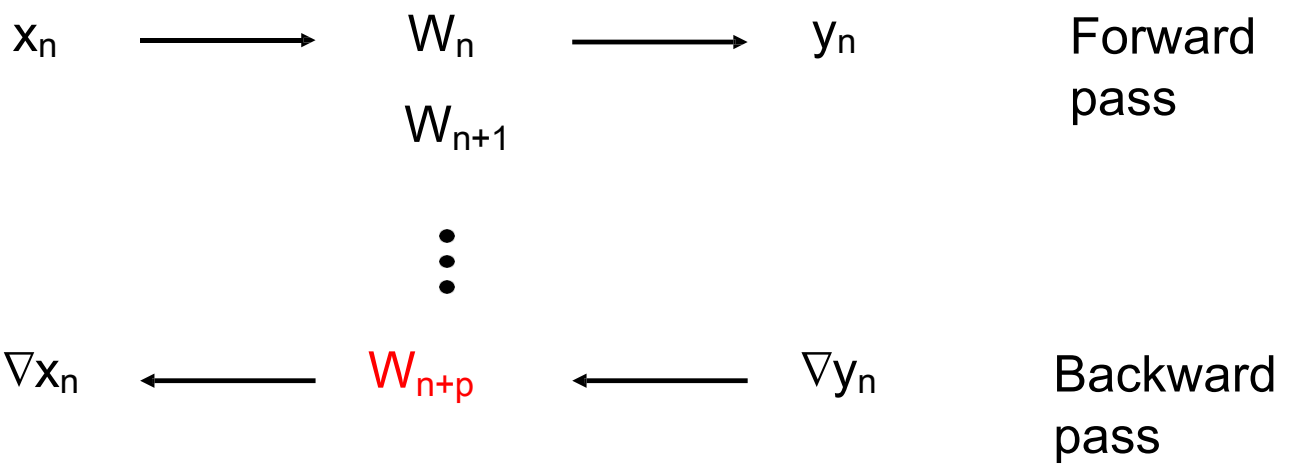
- Background and Motivation

- **Challenges for effective pipeline-parallel training**
  - Partitioning and load balancing operators across workers
  - Scheduling of forward and backward passes of different inputs
  - **Managing weights and activation versions for effective learning**

- Evaluation

Naïve pipelining leads to weight version mismatches

Naïve pipelining leads to **mismatch in weight versions**



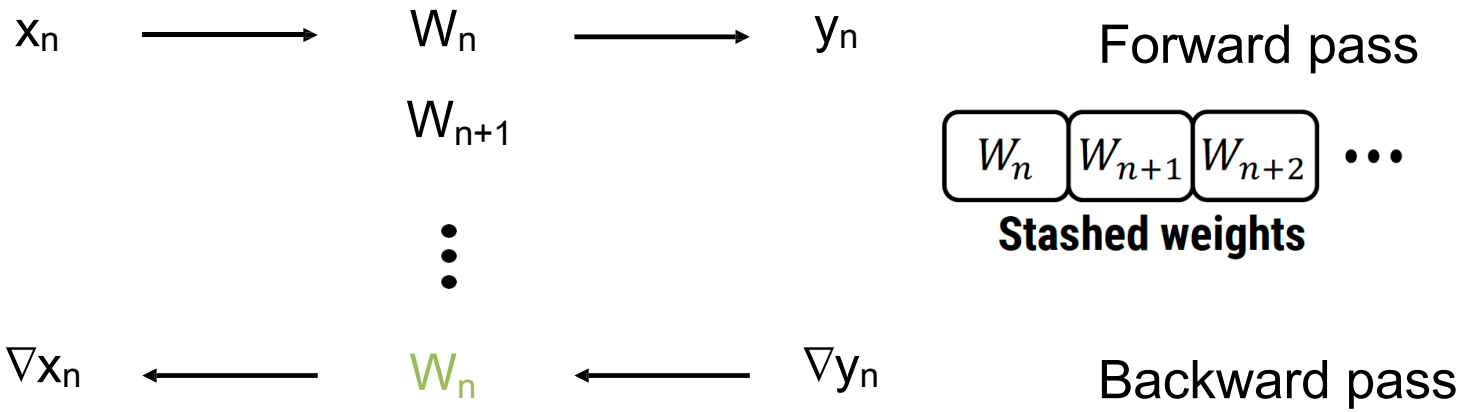
For input n, the forward and backward passes operate on different parameter versions, which leads to incorrect gradient computation.

1F1B Scheduling + Weight Stashing

Naïve pipelining leads to **mismatch in weight versions**

Store **multiple <weight, activation> versions**

- Ensures same weight versions used in both forward and backward pass



- Worst case memory footprint similar to data parallelism $(= n \cdot {}^{(|W|+|A|)}/_{n})$

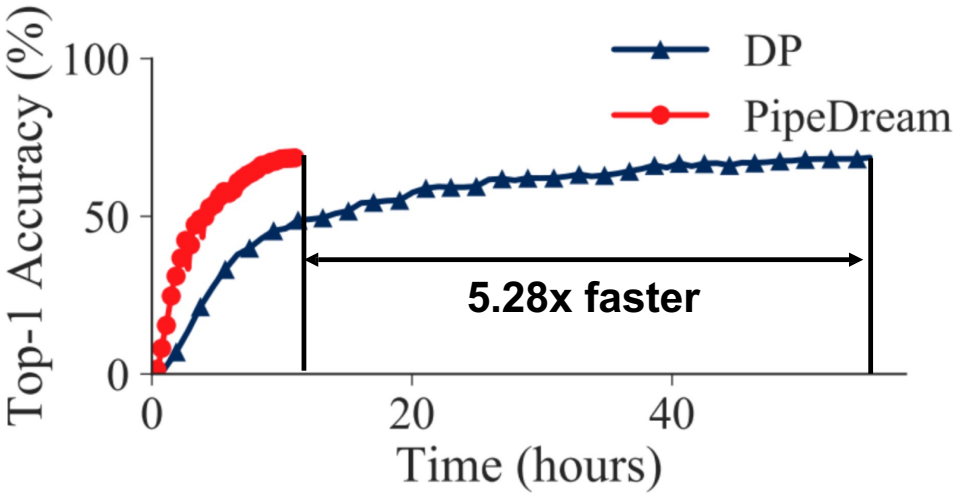
Outline

- Background and Motivation
- Challenges for effective pipeline-parallel training
- **Evaluation**
  - **Setup**
  - **Comparison to Data Parallelism on Time-to-Accuracy**
  - **Communication Overhead of Pipeline Parallelism**
  - **Comparison to Model Parallelism and Hybrid Parallelism on Throughput**
  - **PipeDream's Memory Footprint**

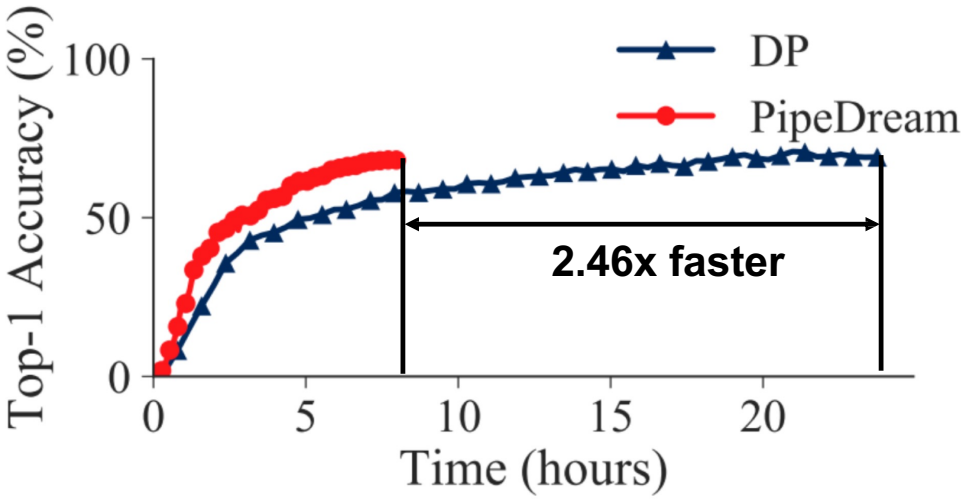
Evaluation Setup

- Integrated PipeDream with PyTorch in ~3000 lines of Python code
- Integrated with PyTorch's communication library
  - NCCL backend for Data Parallelism baselines
  - Gloo backend for PipeDream
- Experiments run on three different server types
  - Cluster A: 4xV100 GPUs, PCIe intra-server, and 10 Gbps inter-server (Azure)
  - Cluster B: 8xV100 GPUs, NVLink intra-server, and 25 Gbps inter-server (AWS)
  - Cluster C: 1xTitan X, and 40 Gbps inter-server (private)

PipeDream > Data Parallelism (DP) end-to-end



(a) Cluster-A.



(b) Cluster-B.

PipeDream vs. Data Parallelism on Time-to-Accuracy

Task	Model	Dataset	Accuracy Threshold	# Servers × # GPUs per server (Cluster)	PipeDream Config	Speedup over DP	
						Epoch time	TTA
Image Classification	VGG-16 [48]	ImageNet [44]	68% top-1	4x4 (A)	15-1	5.28×	5.28×
				2x8 (B)	15-1	2.98×	2.46×
	ResNet-50 [26]	ImageNet [44]	75.9% top-1	4x4 (A)	16	1×	1×
				2x8 (B)	16	1×	1×
	AlexNet [37]	Synthetic Data	N/A	4x4 (A)	15-1	4.92×	N/A
				2x8 (B)	15-1	2.04×	N/A
Translation	GNMT-16 [55]	WMT16 EN-De	21.8 BLEU	1x4 (A)	Straight	1.46×	2.2×
				4x4 (A)	Straight	2.34×	2.92×
				2x8 (B)	Straight	3.14×	3.14×
	GNMT-8 [55]	WMT16 EN-De	21.8 BLEU	1x4 (A)	Straight	1.5×	1.5×
				3x4 (A)	Straight	2.95×	2.95×
				2x8 (B)	16	1×	1×
Language Model	AWD LM [40]	Penn Treebank [41]	98 perplexity	1x4 (A)	Straight	4.25×	4.25×
Video Captioning	S2VT [54]	MSVD [11]	0.294 METEOR	4x1 (C)	2-1-1	3.01×	3.01×

PipeDream vs. Data Parallelism on Time-to-Accuracy

Task	Model	Dataset	Accuracy Threshold	# Servers × # GPUs per server (Cluster)	PipeDream Config	Speedup over DP	
						Epoch time	TTA
Image Classification	VGG-16 [48]	ImageNet [44]	68% top-1	4x4 (A)	15-1	5.28×	5.28×
				2x8 (B)	15-1	2.98×	2.46×
Translation	GNMT-8 [55]	WMT16 EN-De	21.8 BLEU	1x4 (A)	Straight	1.5×	1.5×
				3x4 (A)	Straight	2.95×	2.95×
				2x8 (B)	16	1×	1×
Language Model	AWD LM [40]	Penn Treebank [41]	98 perplexity	1x4 (A)	Straight	4.25×	4.25×
Video Captioning	S2VT [54]	MSVD [11]	0.294 METEOR	4x1 (C)	2-1-1	3.01×	3.01×

Experiments on 4 different tasks: image classification, translation, language modeling, video captioning

PipeDream vs. Data Parallelism on Time-to-Accuracy

Task	Model	Dataset	Accuracy Threshold	# Servers × # GPUs per server (Cluster)	PipeDream Config	Speedup over DP	
						Epoch time	TTA
Image Classification	VGG-16 [48]	ImageNet [44]	68% top-1	4x4 (A)	15-1	5.28×	5.28×
				2x8 (B)	15-1	2.98×	2.46×
	ResNet-50 [26]	ImageNet [44]	75.9% top-1	4x4 (A)	16	1×	1×
				2x8 (B)	16	1×	1×
	AlexNet [25]	Synthetic Data	N/A	4x4 (A)	15-1	4.92×	N/A
				2x8 (B)	15-1	2.04×	N/A
Translation	GNMT-16 [55]	WMT16 EN-De	21.8 BLEU	1x4 (A)	Straight	1.46×	2.2×
				4x4 (A)	Straight	2.34×	2.92×
				2x8 (B)	Straight	3.14×	3.14×
	GNMT-8 [55]	WMT16 EN-De	21.8 BLEU	1x4 (A)	Straight	1.5×	1.5×
				3x4 (A)	Straight	2.95×	2.95×
				2x8 (B)	16	1×	1×
Language Model	AWD LM [40]	Penn Treebank [41]	98 perplexity	1x4 (A)	Straight	4.25×	4.25×
Video Captioning	S2VT [54]	MSVD [11]	0.294 METEOR	4x1 (C)	2-1-1	3.01×	3.01×

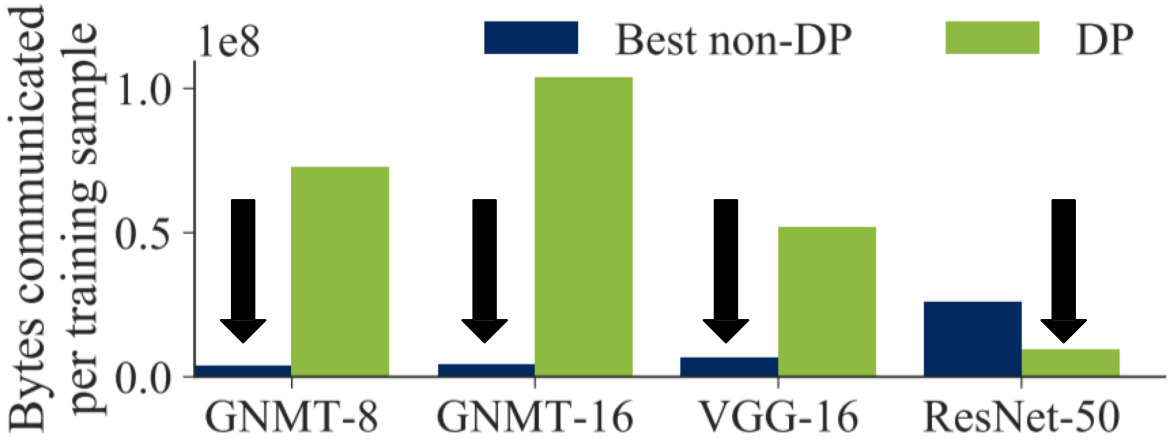
With the same number of GPUs, PipeDream up to 5.3x faster than Data Parallelism

PipeDream vs. Data Parallelism on Time-to-Accuracy

Task	Model	Dataset	Accuracy Threshold	# Servers × # GPUs per server (Cluster)	PipeDream Config	Speedup over DP	
						Epoch time	TTA
	VGG-16 [48]	ImageNet [44]	68% top-1	4x4 (A)	15-1	5.28×	5.28×
				2x8 (B)	15-1	2.98×	2.46×
				4x4 (A)	16	1×	1×
				2x8 (B)	16	1×	1×
				4x4 (A)	15-1	4.92×	N/A
				2x8 (B)	15-1	2.04×	N/A
				4x4 (A)	Straight	1.46×	2.2×
				2x8 (B)	Straight	2.34×	2.92×
				2x8 (C)	Straight	3.14×	3.14×
	GNMT-8 [55]	WMT16 EN-De	21.8 BLEU	1x4 (A)	Straight	1.5×	1.5×
				3x4 (A)	Straight	2.95×	2.95×
				2x8 (B)	16	1×	1×
Language Model	AWD LM [40]	Penn Treebank [41]	98 perplexity	1x4 (A)	Straight	4.25×	4.25×
Video Captioning	S2VT [54]	MSVD [11]	0.294 METEOR	4x1 (C)	2-1-1	3.01×	3.01×

Optimizer recommends a number of different configurations like 15-1, Straight, and a fully data-parallel setup

PipeDream reduces communication overhead



For many models, intermediate activations and gradients order of magnitude smaller than communication with Data Parallelism (DP)

Conclusion

- Model and data parallelism often suffer from **high communication overhead** and **low resource utilization** for certain models and deployments
- PipeDream shows **pipelining** can be used to accelerate DNN training
- Pipelining, when combined with data and model parallelism in a principled way, achieves end-to-end speedups of up to **5.3x**

Code available at
https://github.com/msr-fiddle/pipedream

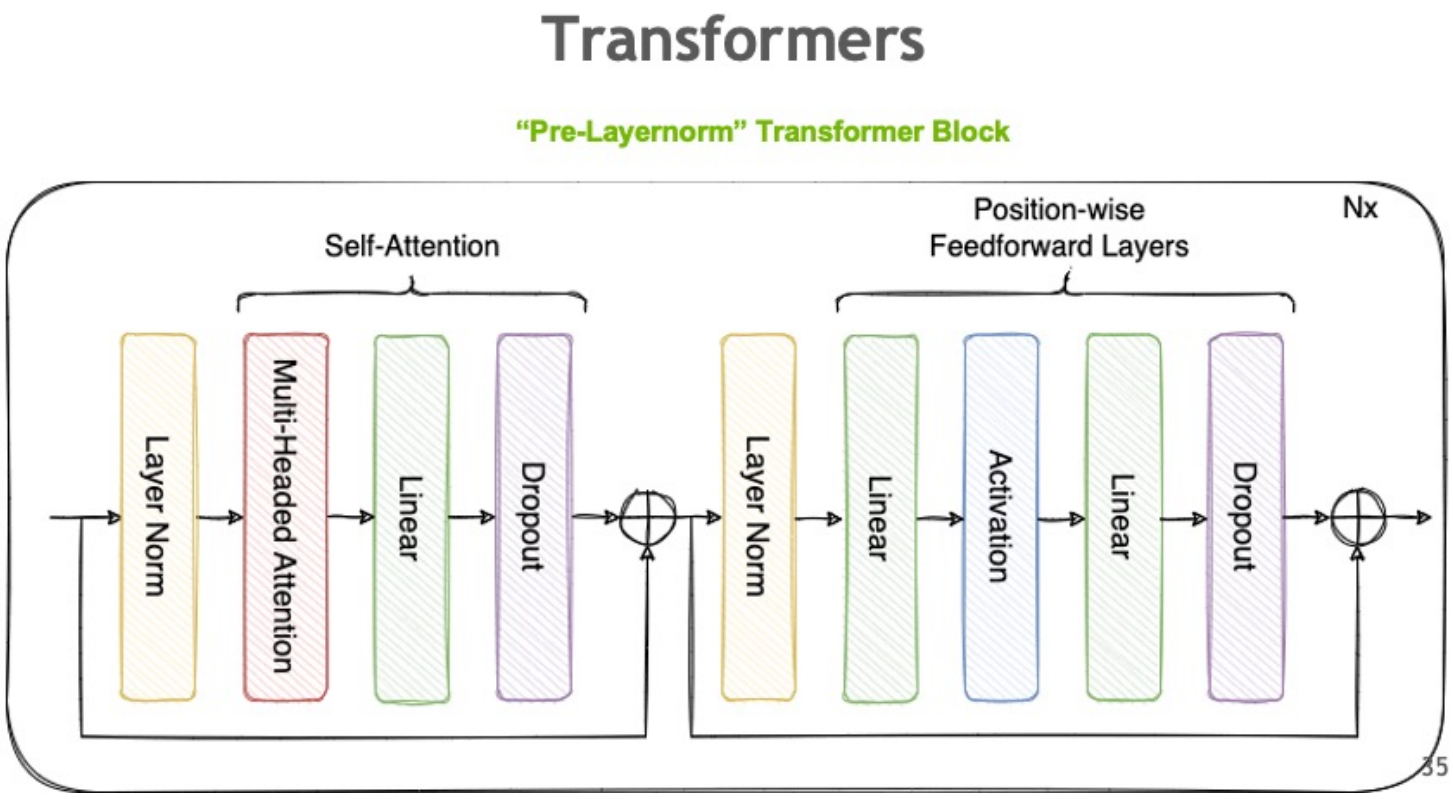


https://cs.stanford.edu/~deepakn/

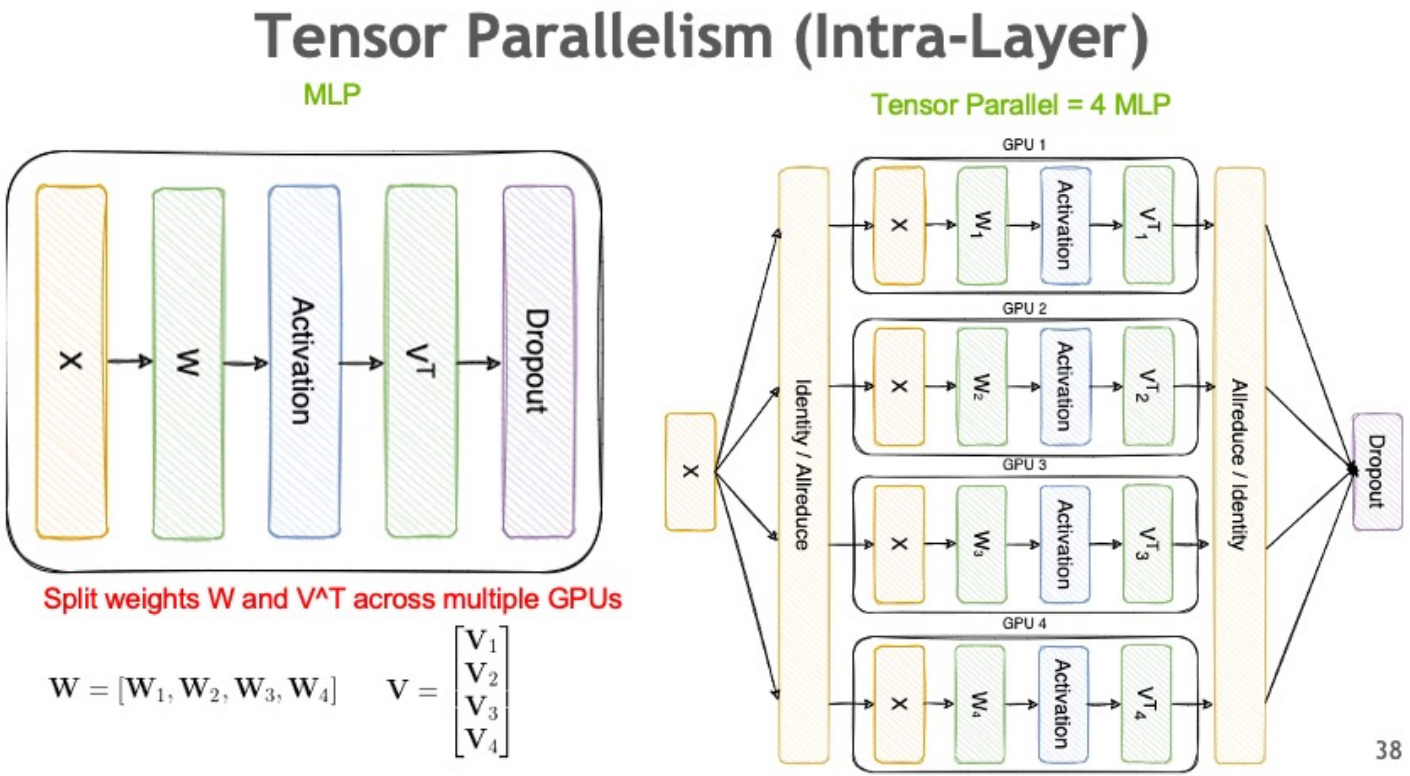
An open question to think about (after class)

- How does PipeDream implement the concurrency and realize precise control of synchronizations?

Another model parallelism: tensor parallelism



Another model parallelism: tensor parallelism



Another model parallelism: expert parallelism

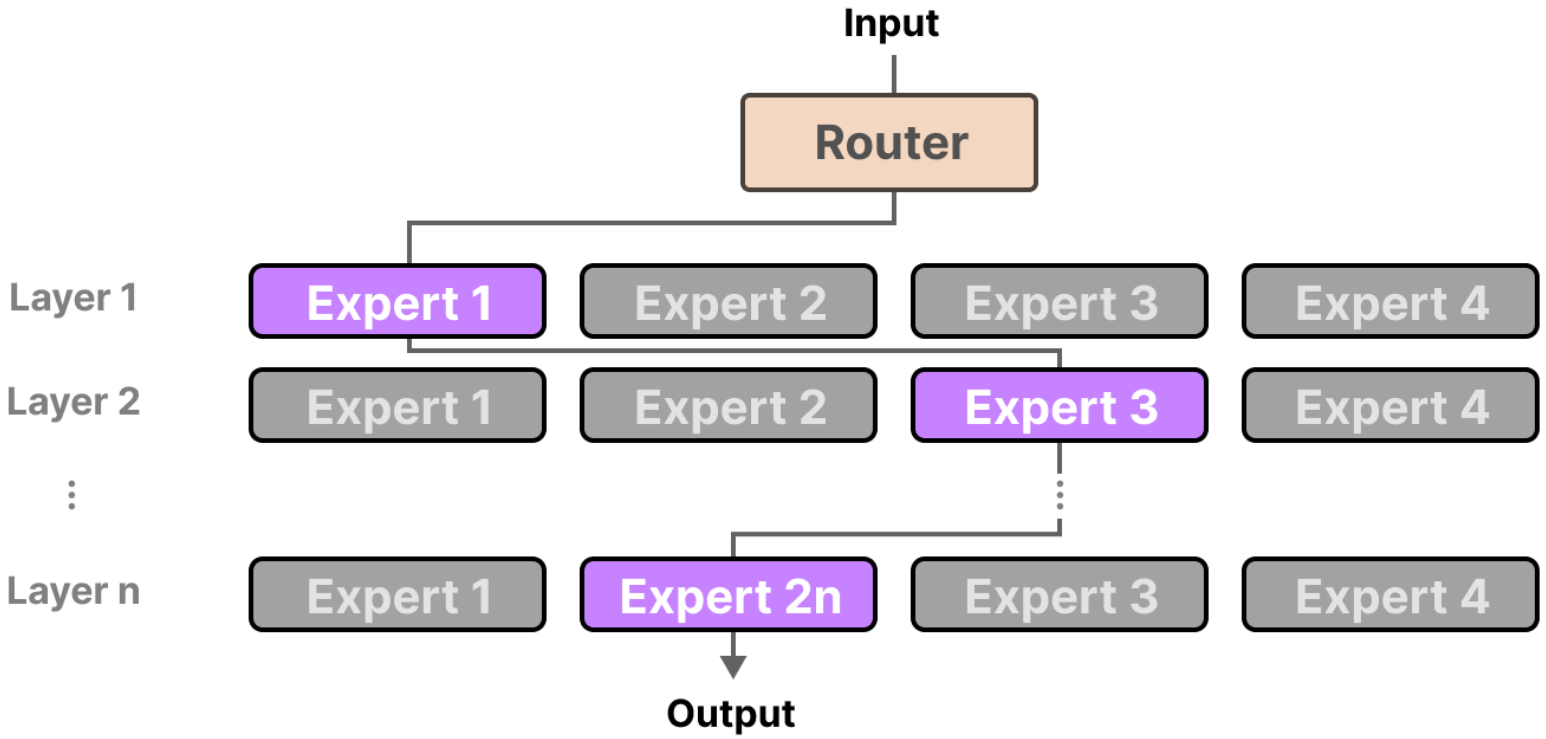


Table of contents

- Modern computer system research venues
- Concurrency issues today
  - **RaceBench**: A Triggerable and Observable Concurrency Bug Benchmark, **CCS'23**
  - [Extended reading] Understanding Real-World Concurrency Bugs in Go, **ASPLOS'19**
- Case study: systems for machine learning/AI
  - **Basics**: neural network, data parallelism, model parallelism
  - **PipeDream**: Generalized Pipeline Parallelism for DNN Training, **SOSP'19**
  - **Orca**: A Distributed Serving System for Transformer-Based Generative Models, **NSDI'22**
  - **DistServe**: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving, **OSDI'24**

Into the era of LLMs

- Yu et al. “Orca: A Distributed Serving System for Transformer-Based Generative Models”, in OSDI’22

- https://www.usenix.org/conference/osdi22/presentation/yu

Table of contents

- Modern computer system research venues
- Concurrency issues today
  - **RaceBench**: A Triggerable and Observable Concurrency Bug Benchmark, **CCS'23**
  - [Extended reading] Understanding Real-World Concurrency Bugs in Go, **ASPLOS'19**
- Case study: systems for machine learning/AI
  - **Basics**: neural network, data parallelism, model parallelism
  - **PipeDream**: Generalized Pipeline Parallelism for DNN Training, **SOSP'19**
  - **Orca**: A Distributed Serving System for Transformer-Based Generative Models, **NSDI'22**
  - **DistServe**: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving, **OSDI'24**







DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving

Yinmin Zhong Shengyu Liu Junda Chen Jianbo Hu
Yibo Zhu Xuanzhe Liu Xin Jin Hao Zhang

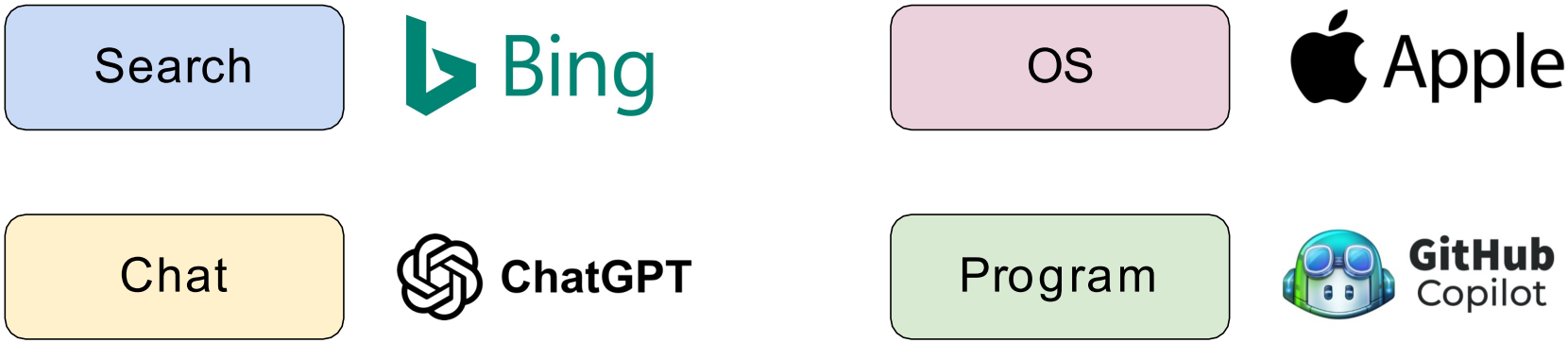


北京大学
PEKING UNIVERSITY



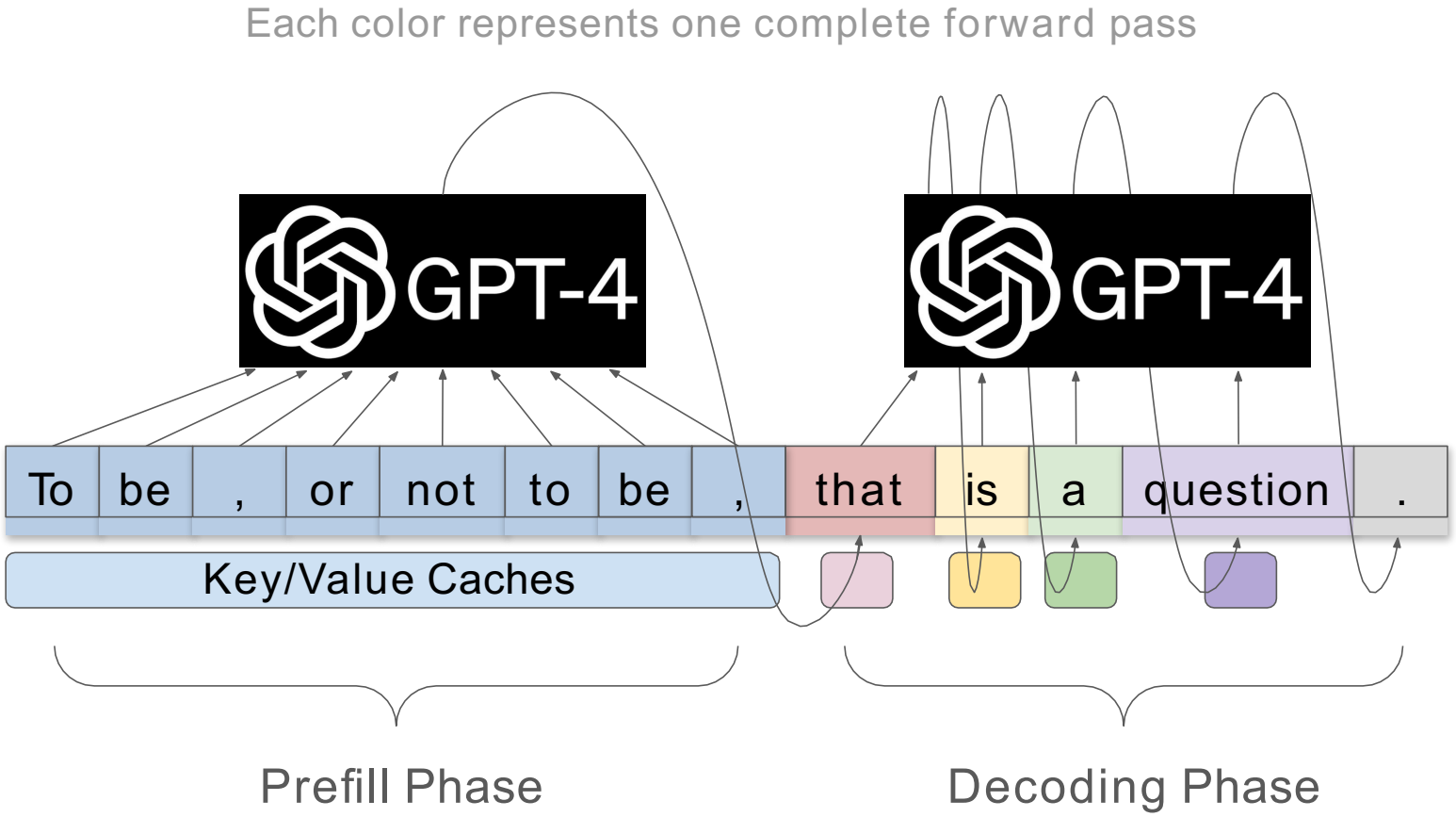



Various applications are powered by LLMs

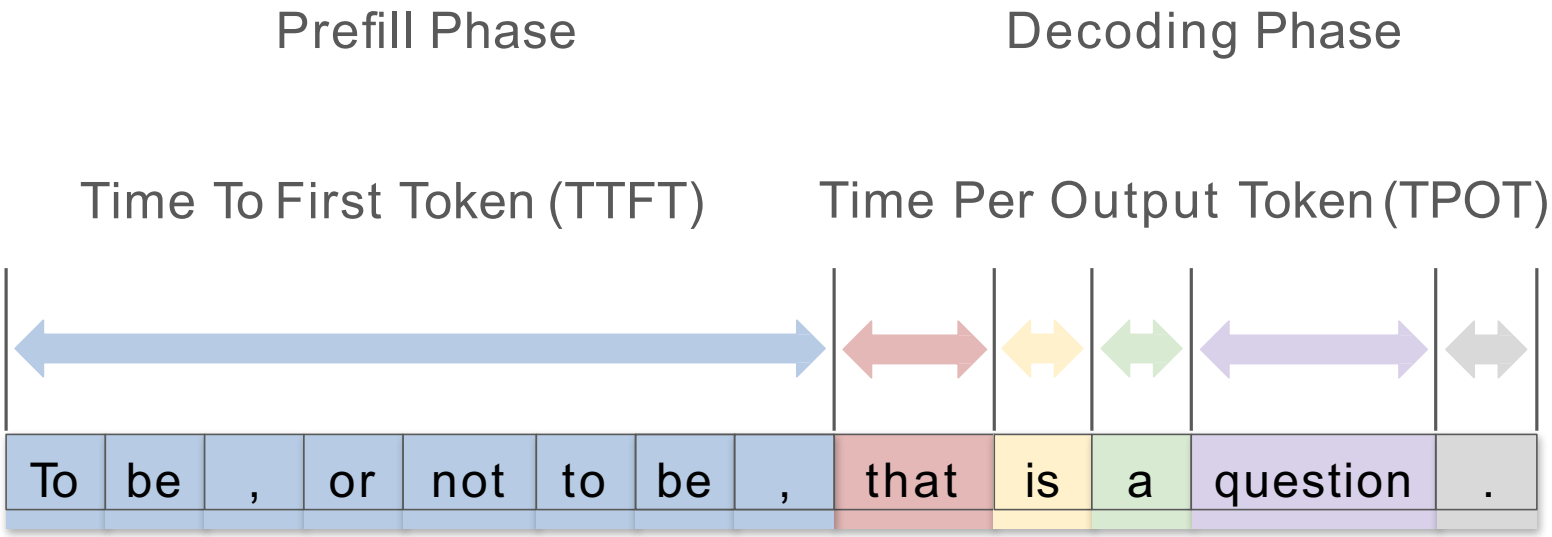




LLM Inference



LLM Inference Latency



Latency = TTFT + TPOT * #Token

Different apps have various latency requirements

	TTFT	TPOT
Chat	Low (< 1s)	Match read speed (~ 100ms)
Search	Very Low (~ 200ms)	Match read speed (~ 100ms)
Program	Very Low (~ 200ms)	Very Low (~ 50ms)

Our Goal

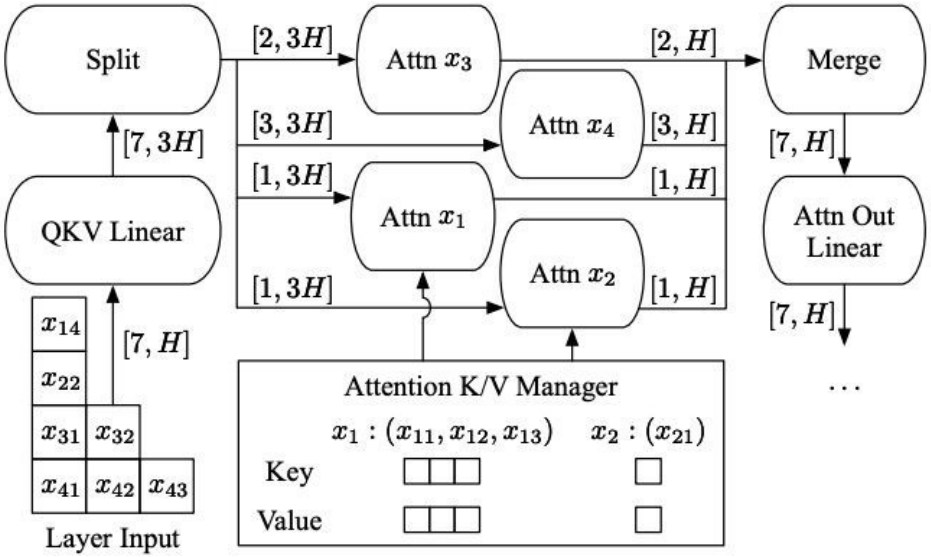
Use the fewest #GPUs to serve as many requests as possible while subject to various latency requirements of apps.

- *Service Level Objective (SLO)*
- *SLO Attainment*
- *Per-GPU Goodput*

Formally, our goal is to maximize the Per-GPU Goodput

Existing systems maximize the overallthroughput

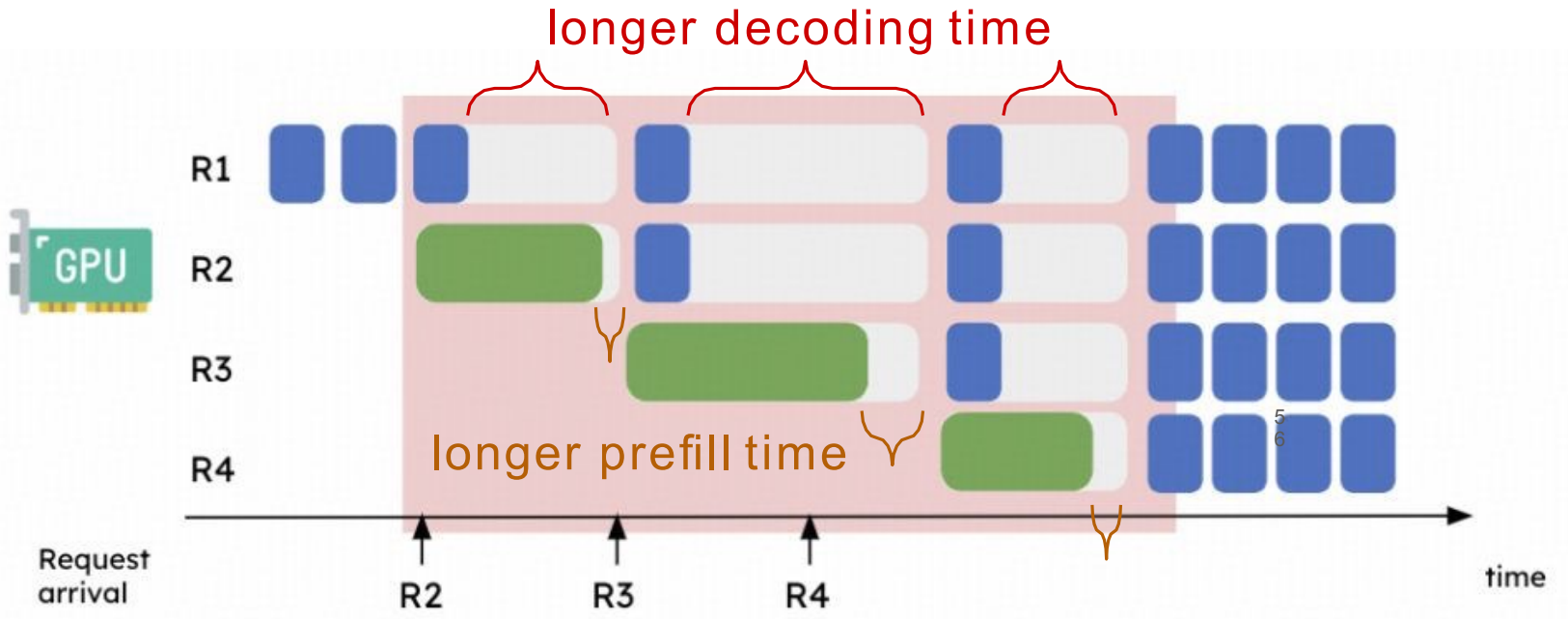
Existing approach: Batch requests in prefill and decoding phase together



Source: Gyeong-In Yu, et al. “Orca: A Distributed Serving System for Transformer-Based Generative Models”

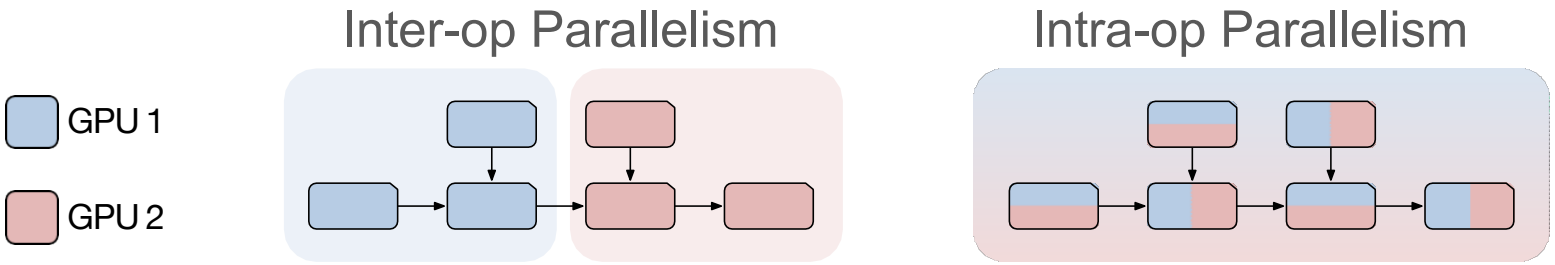
Problem 1: Prefill-Decoding Interference

Batching prefill and decoding phase together hurt both TTFT and TPOT

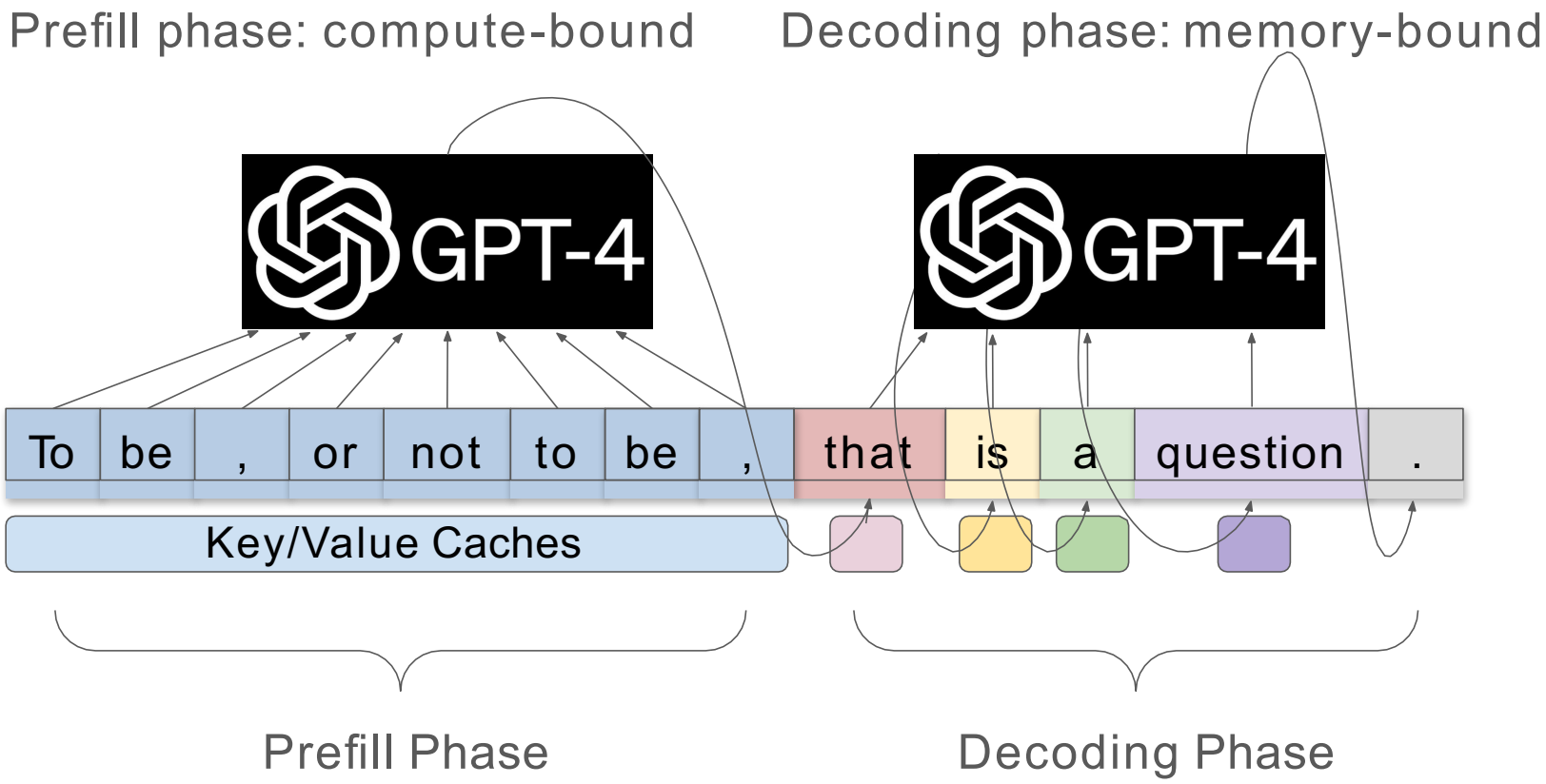


Problem 2: Resource and Parallelism Coupling

Batching the two phases make them share the same parallel strategy

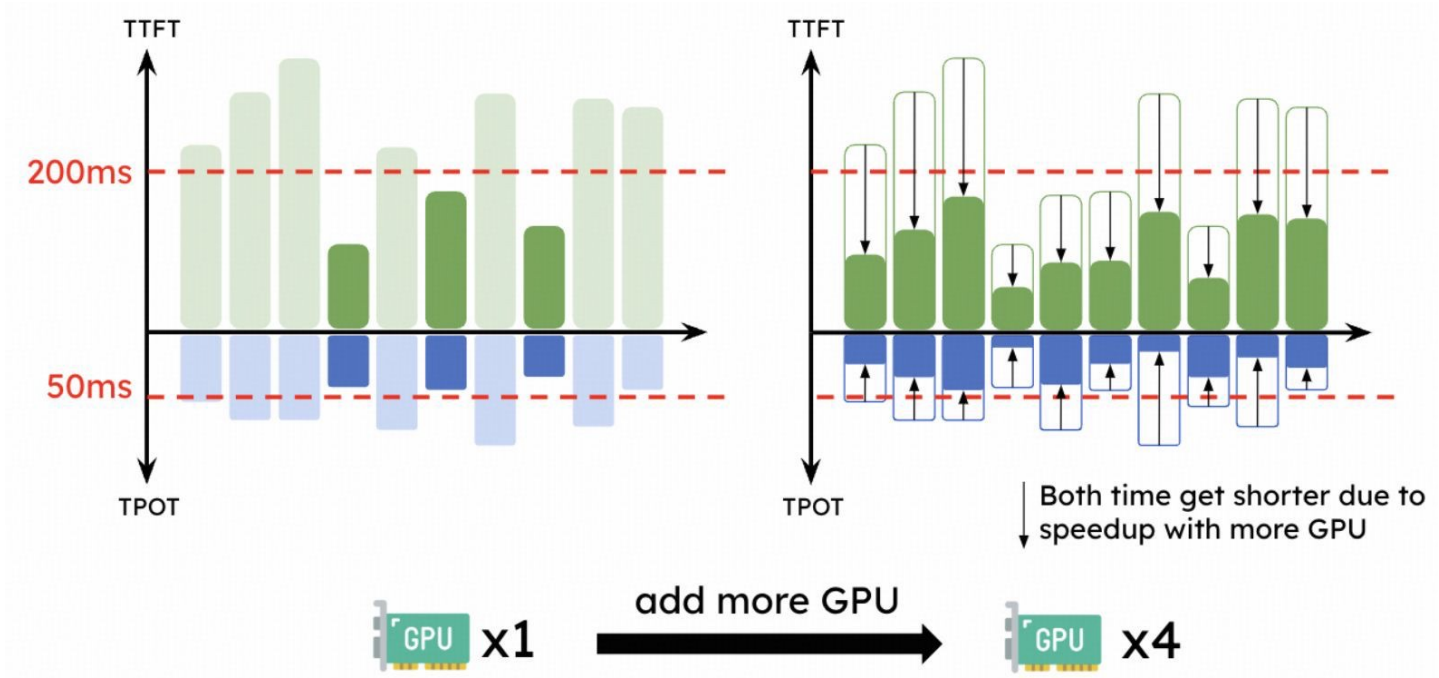


Problem 2: Resource and Parallelism Coupling



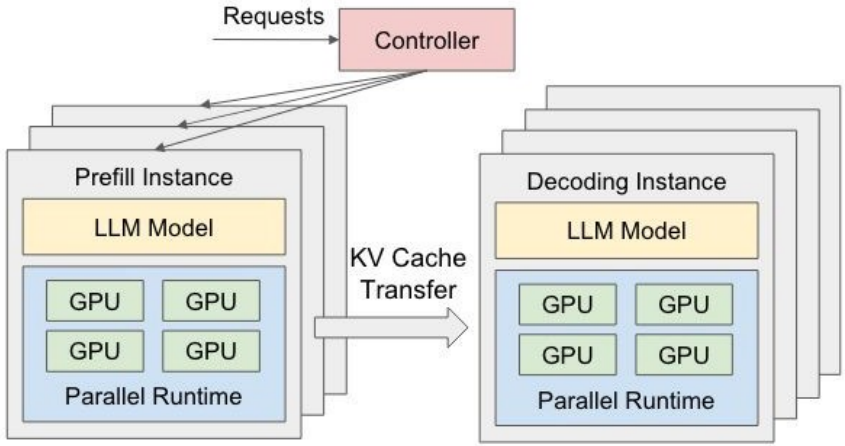
Problem 2: Resource and Parallelism Coupling

Coupling leads to overprovision resources to meet the more demanding SLO

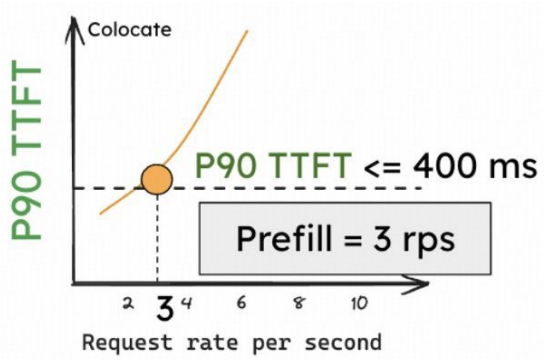


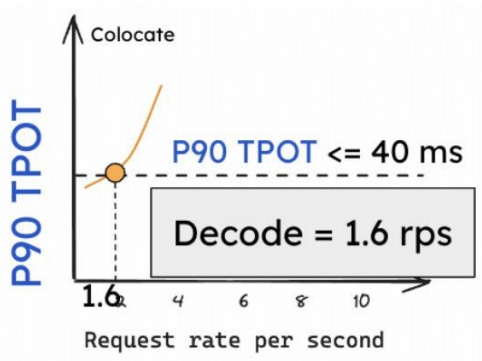
Opportunity: Disaggregting Prefill and Decoding

- Prefill-Decoding interference is immediately eliminated
- Naturally divide the SLO satisfaction problem into two optimizations:
  - Prefill instance optimizes for TTFT.
  - Decoding instance optimizes for TPOT.
  - Choose the most suitable parallelism and resource allocation for each phase.

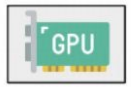


Opportunity: Disaggregting Prefill and Decoding





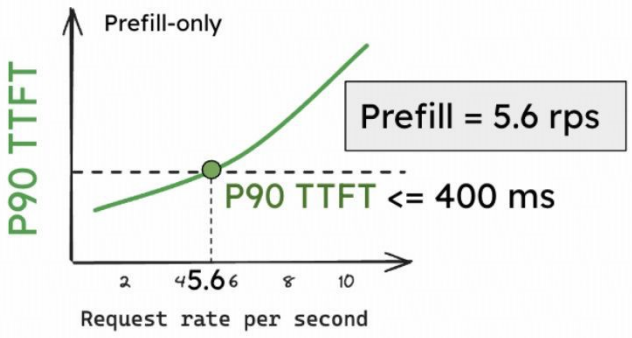
Colocation

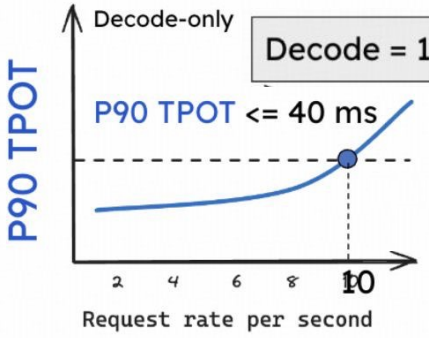


Max System rps

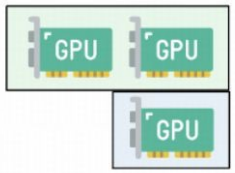
= Min(Prefill, Decode)

= 1.6 rps / GPU





Disaggregation (2P1D)



Max System rps

= Min (5.6 x **2**, 10) rps / 3 GPU

= 3.3 rps / GPU

Challenges of Disaggregation

- Communication overhead for KV-Cache transmission
- The optimization target — ***per-GPU goodput***, is difficult to optimize:
  - the workload pattern
  - SLO requirements
  - parallelism strategies
  - resource allocation
  - network bandwidth

DistServe Overview

Definition of Placement:

(1) parallelism strategy for prefill/decoding instance
(2) the number of each instance to deploy
(3) how to place them onto the physical cluster

Featured algorithms:

- Placement for High Node-Affinity Cluster
- Placement for Low Node-Affinity Cluster
- Online Scheduling Optimization

Placement for High Node-Affinity Cluster

Assumption:

- Nodes are connected with high bandwidth network, e.g., InfiniBand.

Observation:

- We can optimize prefill and decoding instances separately.

Algorithm Sketch:

- Use simulation to measure the goodput for a specific parallelism config.
- Obtain the optimal parallelism config for each phase.
- Use replication to match the overall traffic.

Placement for Low Node-Affinity Cluster

Assumption:

- GPUs inside one node are connected with NVLINK.

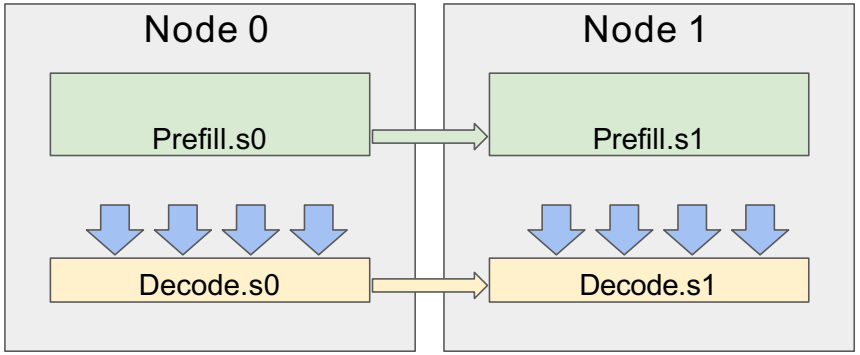
Observation:

- KV-Cache transmission only happens between ***the same layer.***

Algorithm Sketch:

- Similar to the previous one but add the constraint to require the same stage of prefill/decoding instances to be on the same node.

Example:



Online Scheduling Optimization

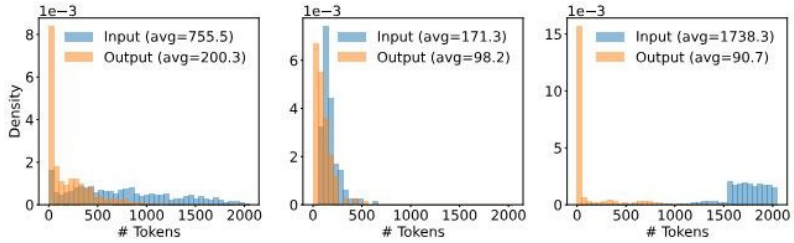
- Scheduling to reduce pipeline bubbles
- Combat workload burstiness
- Periodic Replaning

Evaluations

- Setup:
  - 4 x DGX-A100, each with 8 x NVIDIA A100-80GB
  - intra-node network bandwidth: 600GB/s
  - cross-node network bandwidth 25Gbps
- Workloads:

Application	Model Size	TTFT	TPOT	Dataset
Chatbot OPT-13B	26GB	0.25s	0.1s	ShareGPT [8]
Chatbot OPT-66B	132GB	2.5s	0.15s	ShareGPT [8]
Chatbot OPT-175B	350GB	4.0s	0.2s	ShareGPT [8]
Code Completion OPT-66B	132GB	0.125s	0.2s	HumanEval [14]
Summarization OPT-66B	132GB	15s	0.15s	LongBench [13]

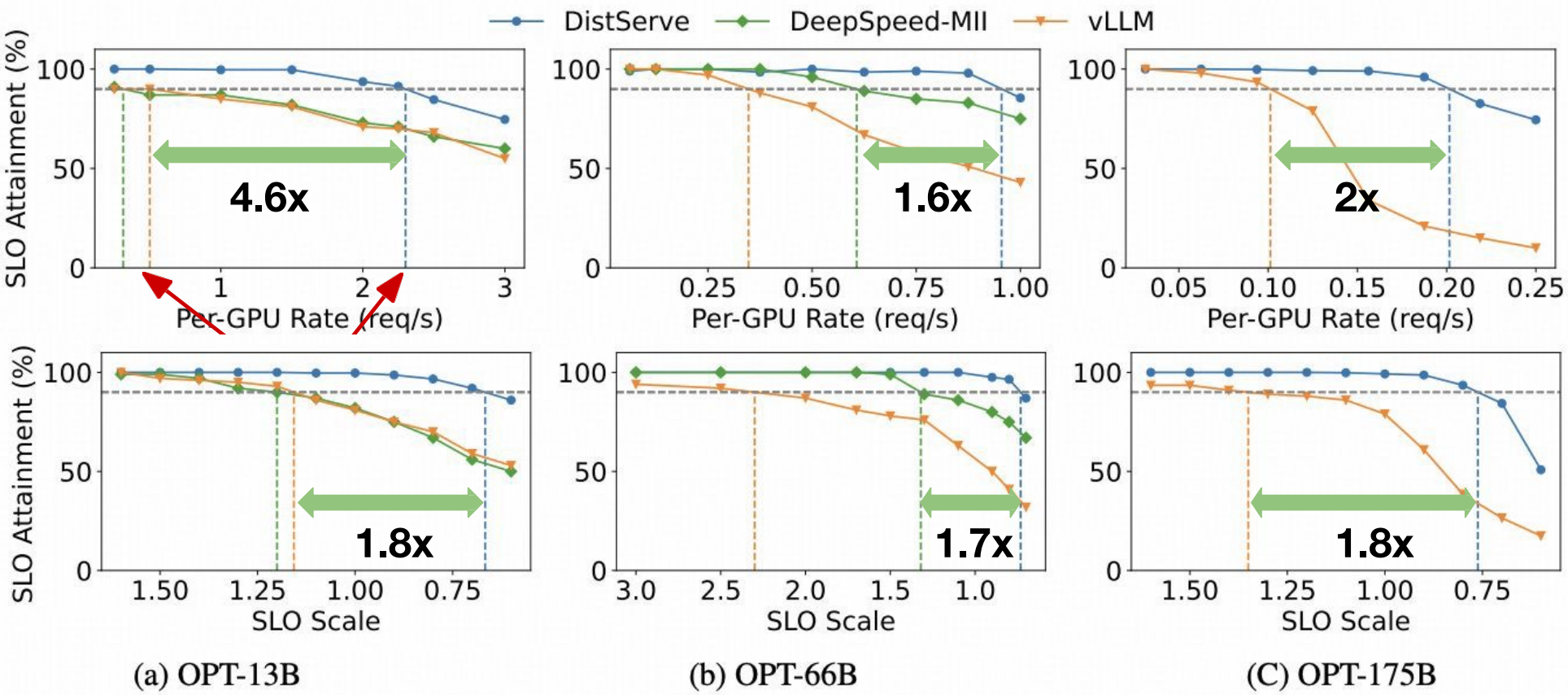
Table 1: Workloads in evaluation and latency requirements.



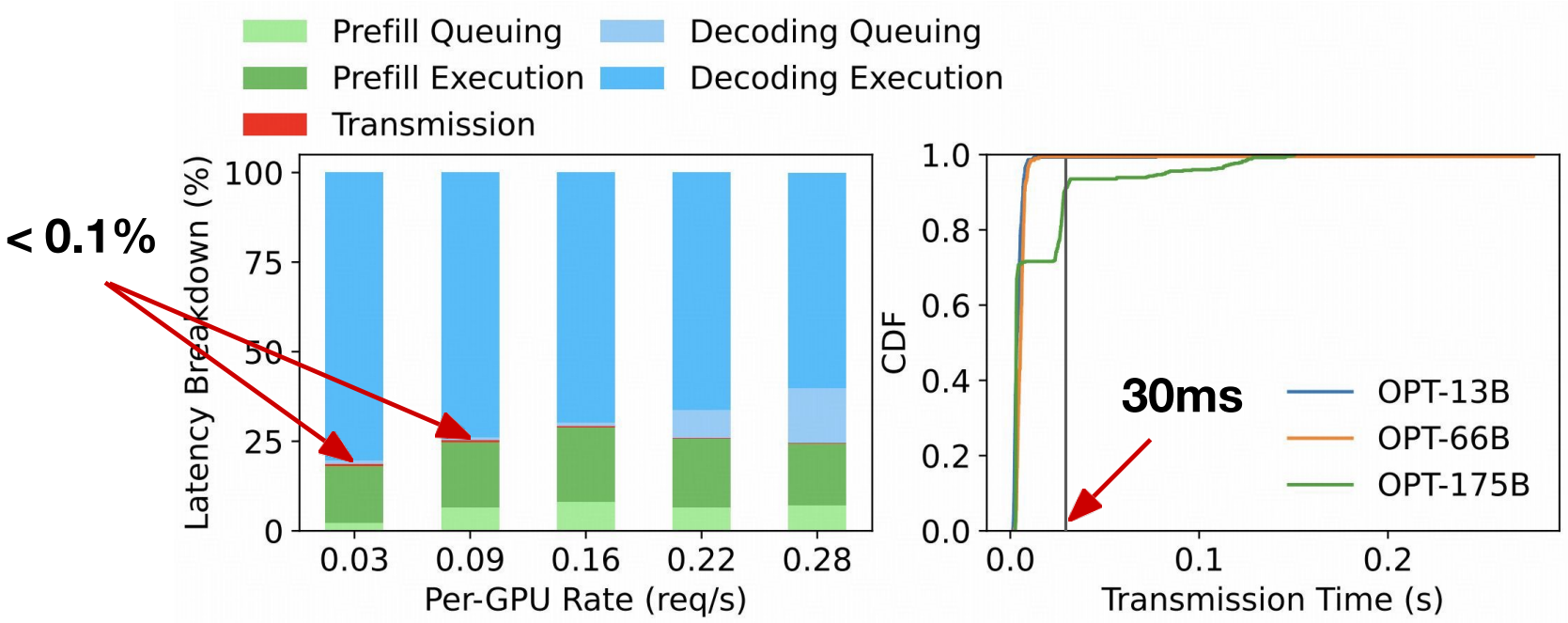
(a) ShareGPT (b) HumanEval (c) LongBench

- Metric: SLO Attainment
- Baseline: vLLM, DeepSpeed-MII

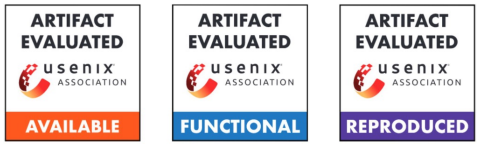
End-to-end Experiment –Chatbot



Latency Breakdown



Summary



- DistServe disaggregates prefill from decoding to serve LLM inference:
  - Bandwidth-aware placement algorithm to minimize communication overhead
  - Co-optimize the parallelism strategies for prefill/decoding instances
  - Online scheduling to handle real-world workload
- Experiments show that DistServe can serve 7.4x more requests or 12.6x tighter SLO, compared to SOTA systems, while staying within latency requirements for > 90% of requests.

https://github.com/LLMServe/DistServe

One last note…

- Operating systems in the age of AI
- Everything gets reinvented → this class more relevant than ever

The Bitter Lesson, by Rich Sutton

The Bitter Lesson

Rich Sutton Turing Award winner (2024)

March 13, 2019

The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin. The ultimate reason for this is Moore's law, or rather its generalization of continued exponentially falling cost per unit of computation. Most AI research has been conducted as if the computation available to the agent were constant (in which case leveraging human knowledge would be one of the only ways to improve performance) but, over a slightly longer time than a typical research project, massively more computation inevitably becomes available. Seeking an improvement that makes a difference in the shorter term, researchers seek to leverage their human knowledge of the domain, but the only thing that matters in the long run is the leveraging of computation. These two need not run counter to each

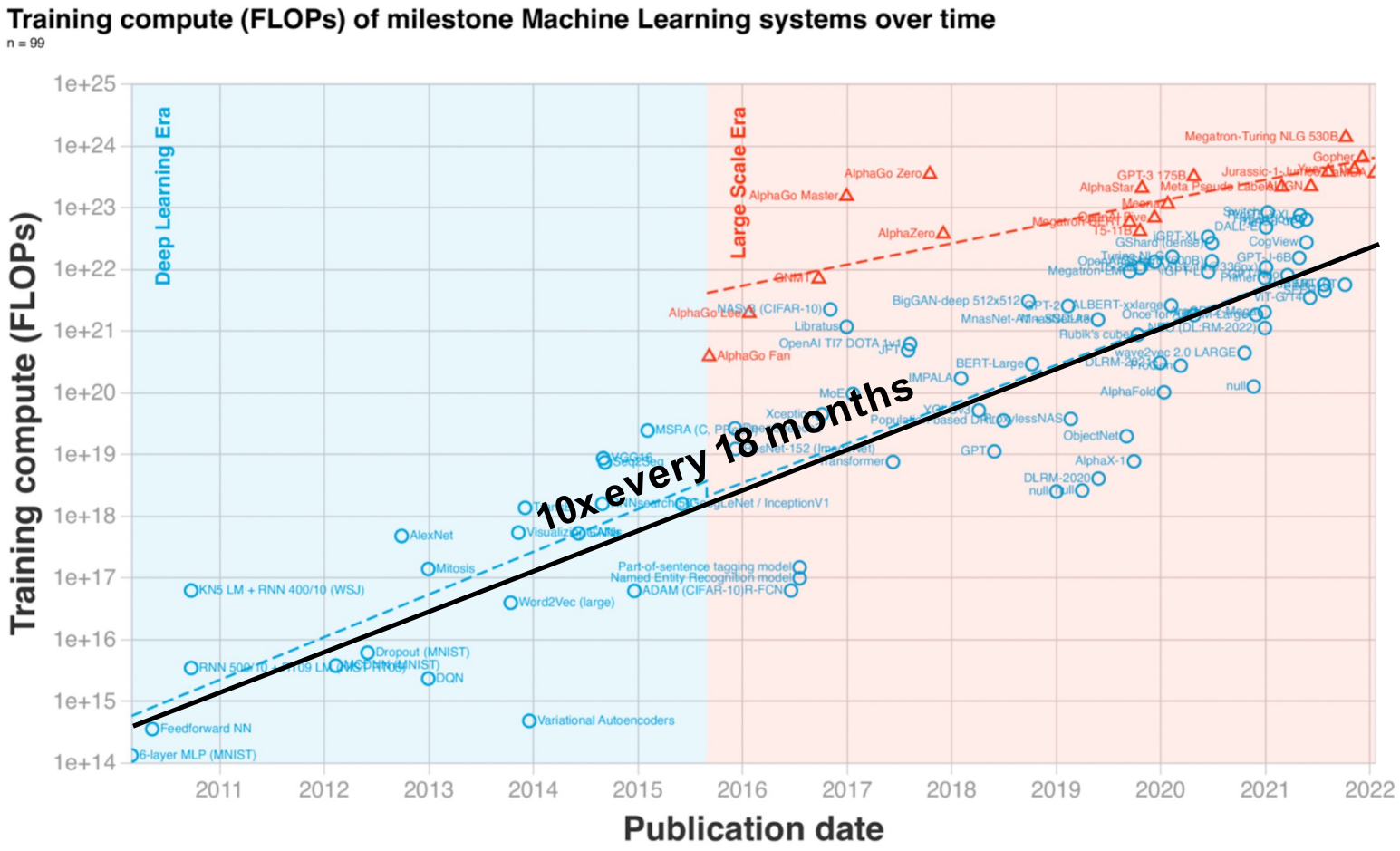
- https://www.cs.utexas.edu/~eunsol/courses/data/bitter_lesson.pdf

The Bitter Lesson, by Rich Sutton

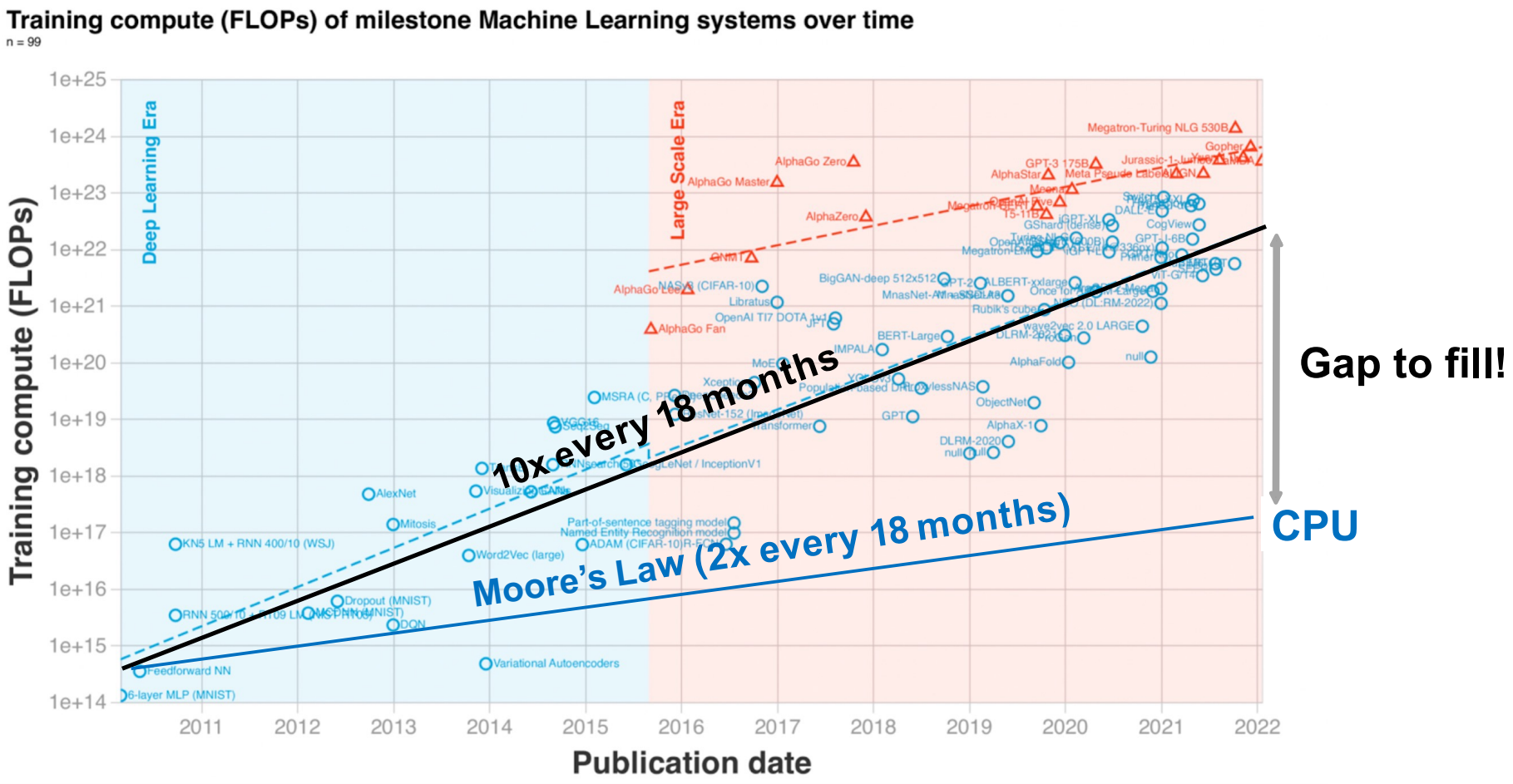
- Human-designed knowledge does not scale
  - Handcrafted rules eventually plateau
- Learning and search scale with computation
  - This pattern underlies AlphaGo, deep learning, and LLMs
- Systems unlock that computation
  - Better scheduling, memory, parallelism, and networking enable larger-scale, smarter AI

Takeaway: More efficient computation creates more room for intelligence.

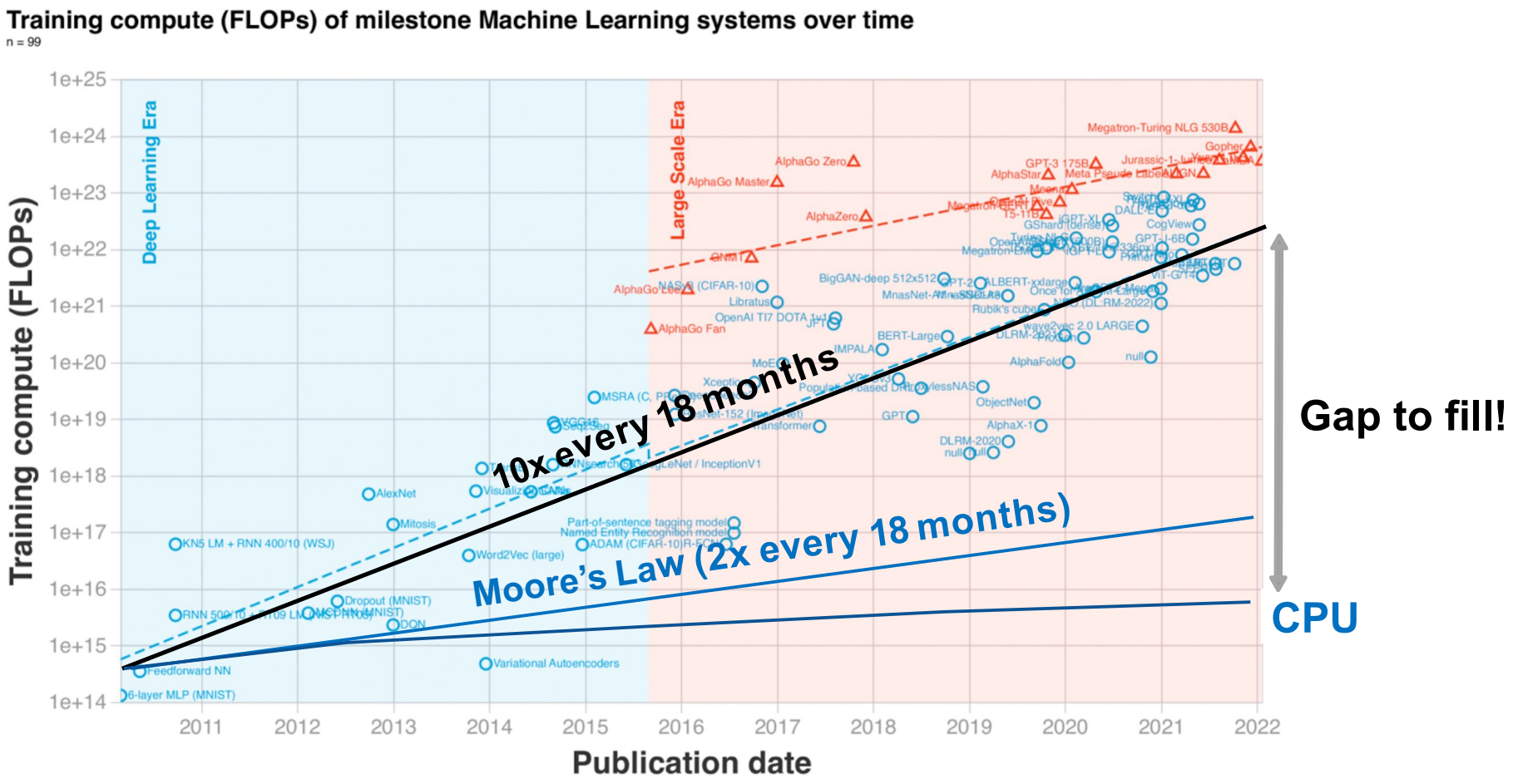
AI demands are exploding



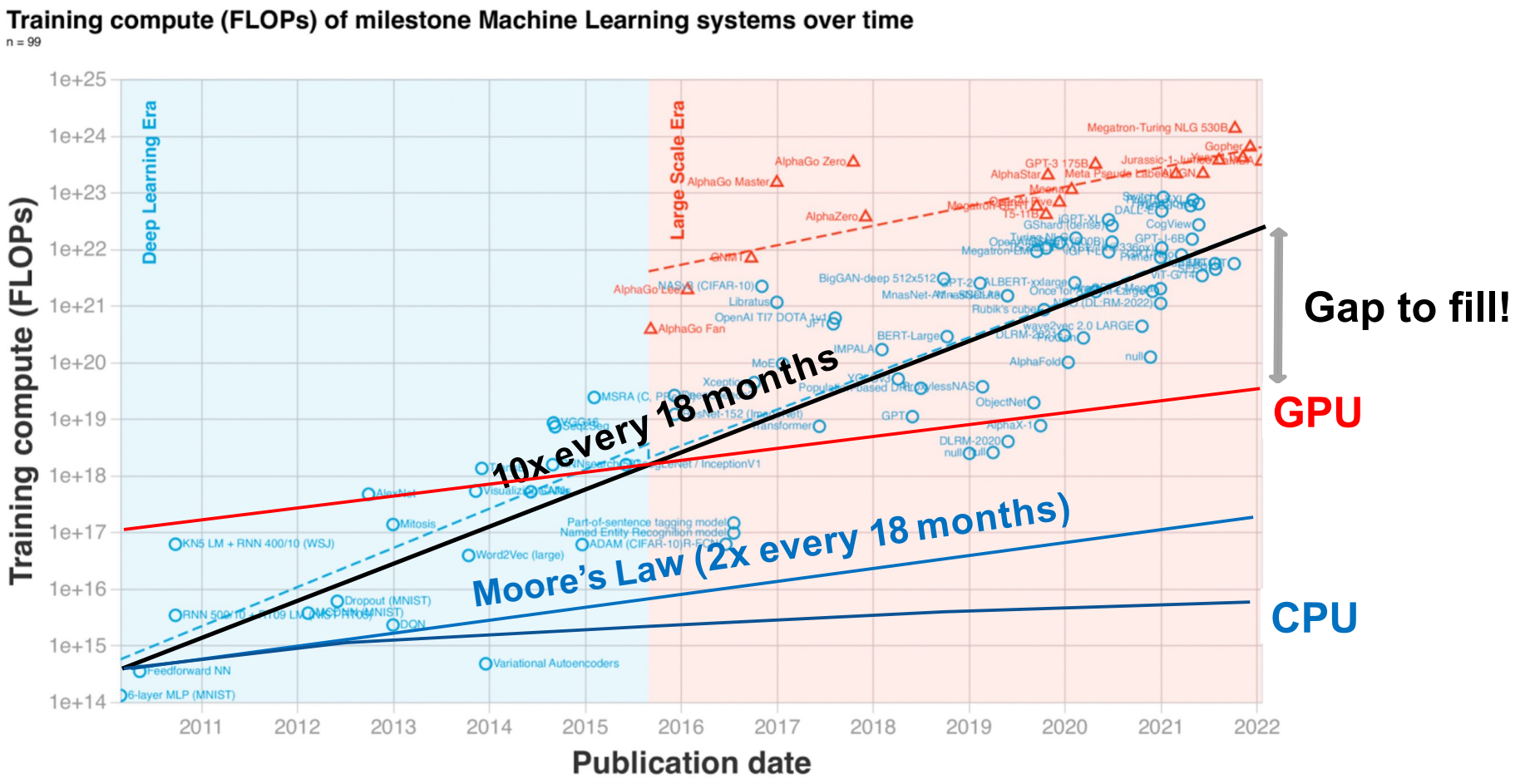
Growing gap between demand and supply



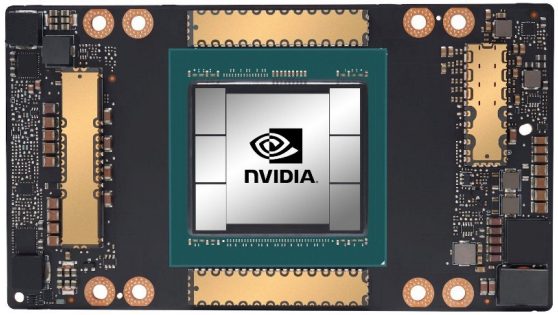
Growing gap between demand and supply

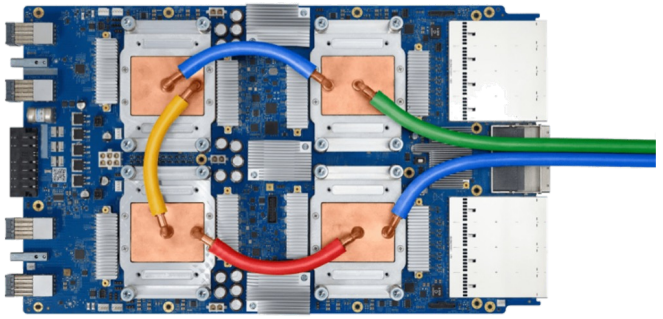


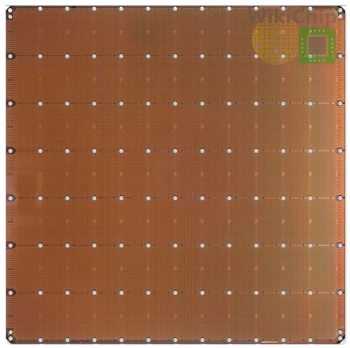
Specialized hardware not enough

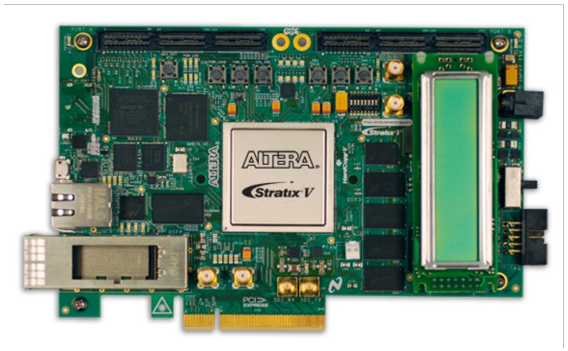


Specialized processors to the rescue → more complexity





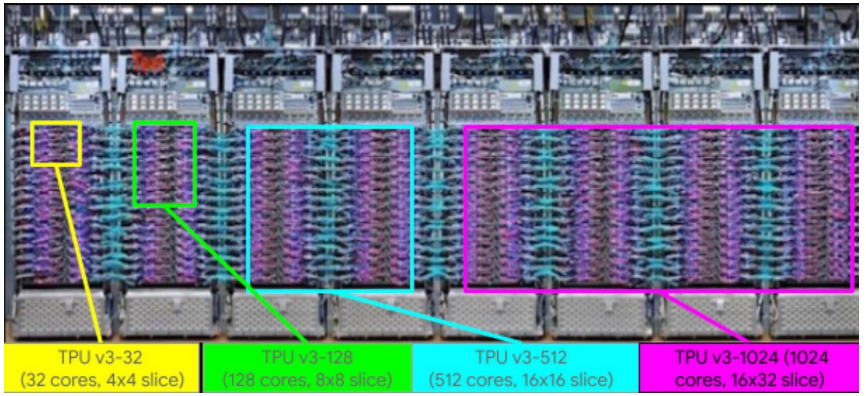






From servers to pods → more complexity







Your feedback is welcomed!



- https://forms.cloud.microsoft/r/Q6szydQLXY

Thanks